Email: ahds@engineer.com

SCR Engineering Simulation

MATLAB Online | Illustrated Engineering Publication | V6

65 numbered engineering figures • fully commented MATLAB source • reproduction instructions

AHDS — Anissoff Hardware Design System | Design once. Reuse forever.

Publication purpose

This publication is a navigable engineering companion to the AHDS SCR V6 handbook. It assembles the complete set of 65 existing reference figures, explains the nine model families, documents the source-code workflow and gives practical instructions to reproduce the graphs using MATLAB Online.

Author-verified execution: The project author reports successfully running and testing the complete SCR V6 MATLAB program in MATLAB Online and generating all 65 engineering graphs. The 65 illustrations reproduced in this publication are existing reference images from the AHDS V6 handbook, not exports made in a new session for preparing this edition. The mathematical models are educational abstractions, not measured device data or hardware certification.

Contents

1. MATLAB Online: setup and execution

2. Model definitions, parameters and interpretation

3. Illustrated atlas: SCR_001 through SCR_065

4. Reproducibility, limitations and validation checklist

Appendix A. Complete commented MATLAB source

Appendix B. Package contents and supporting references

1. MATLAB Online: setup and execution

1. Open https://matlab.mathworks.com/ and sign in to a MathWorks account. MATLAB Online access and licensing depend on the account.

2. Upload 01_MATLAB_Source/SCR11_v6_REVISED_commented.m to the MATLAB Online Files panel.

3. For a first check set FIGURE_COUNT = 3; for the complete result use FIGURE_COUNT = 65. Set FIG_VISIBLE = off for batch export; on displays the figures interactively.

4. In the Editor choose Run, or execute run('SCR11_v6_REVISED_commented.m') from the correct folder.

5. Check the Command Window for the final delivery path of AHDS_SCR_65_Graphs.zip. Download that file from the Files panel.

6. Extract the generated archive and check for SCR_001.png through SCR_065.png, plus RUN_REPORT.txt with STATUS: COMPLETE.

2. Simulation families: source-derived descriptions

Figures	Model family	Interpretation
001–006	SCR on-state V–I	Illustrative voltage threshold +

		linear conductance; varying current range. Not a measured SCR characteristic.
007–013	Gate excitation and latching	Rectangular gate pulse, normalized anode-current rise and a chosen latching threshold.
014–021	Phase-controlled resistive load	Ideal half-wave rectifier for different firing angles with pure resistive load.
022–031	Inductive R–L load	Analytical SCR current and first natural extinction after a firing event.
032–041	R–L–C transient	Series RLC initial capacitor discharge used as a simplified snubber proxy.
042–048	Turn-off margin	Available reverse-bias duration compared with assumed turn-off time tq.
049–055	Lumped junction thermal response	First-order thermal resistance/capacitance model and illustrative power loss.
056–061	Prospective fault I^2t	DC-step R–L fault current integrated over time; not a sinusoidal AC fault model.
062–065	Extinction-angle sensitivity	Numerically detected extinction angle beta against firing angle alpha at varied resistance.

Default main AC parameters in the source include 230 Vrms at 50 Hz, with R = 20 Ω and L = 100 mH in the core R–L examples. Other parameters vary by model family; inspect the commented script before changing them.

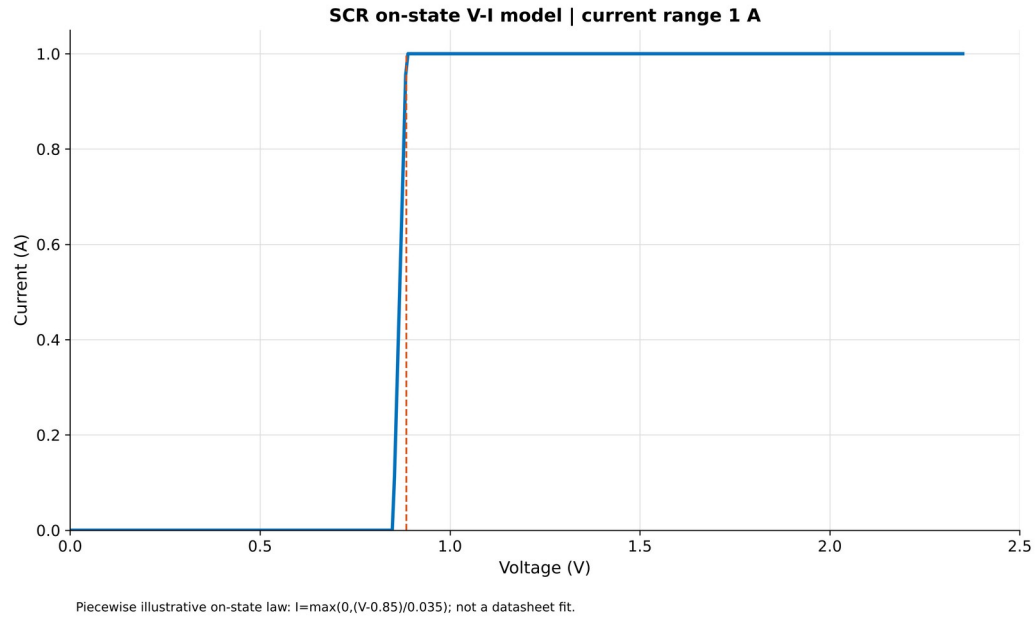
3. Illustrated atlas — all 65 reference figures

The 65 illustrations below preserve the original figure sequence. Image names ending in _REFERENCE are from the existing reviewed handbook. MATLAB Online will create corresponding SCR_###.png files if the script executes successfully.

001–006 | SCR on-state V–I

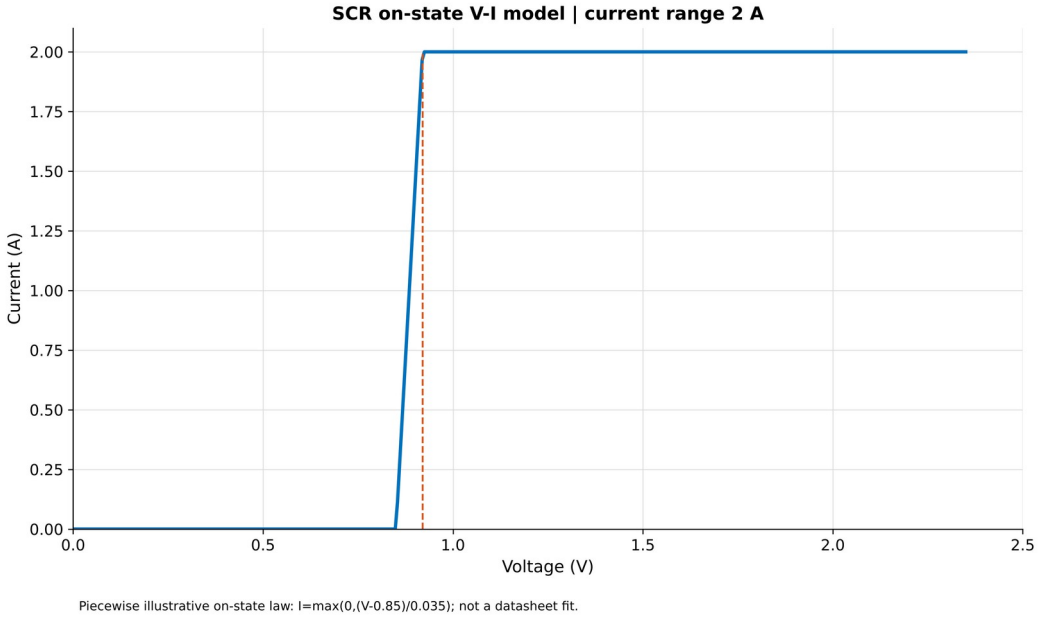
Illustrative voltage threshold + linear conductance; varying current range. Not a measured SCR characteristic.

Figure SCR_001 — SCR on-state V–I



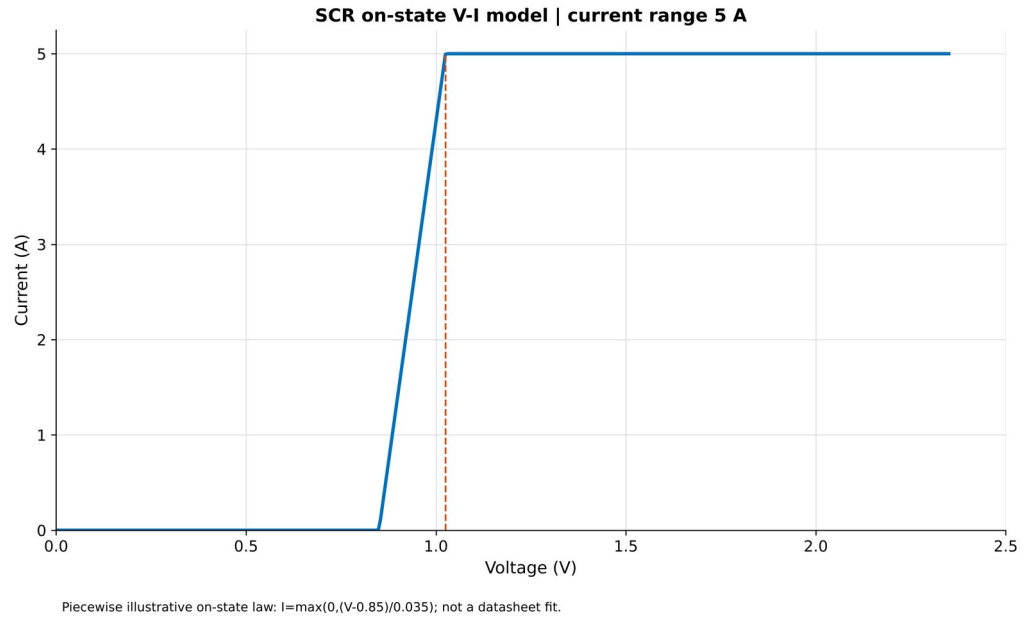
REFERENCE illustration SCR_001. Reproduce as SCR_001.png with the supplied MATLAB script.

Figure SCR_002 — SCR on-state V–I



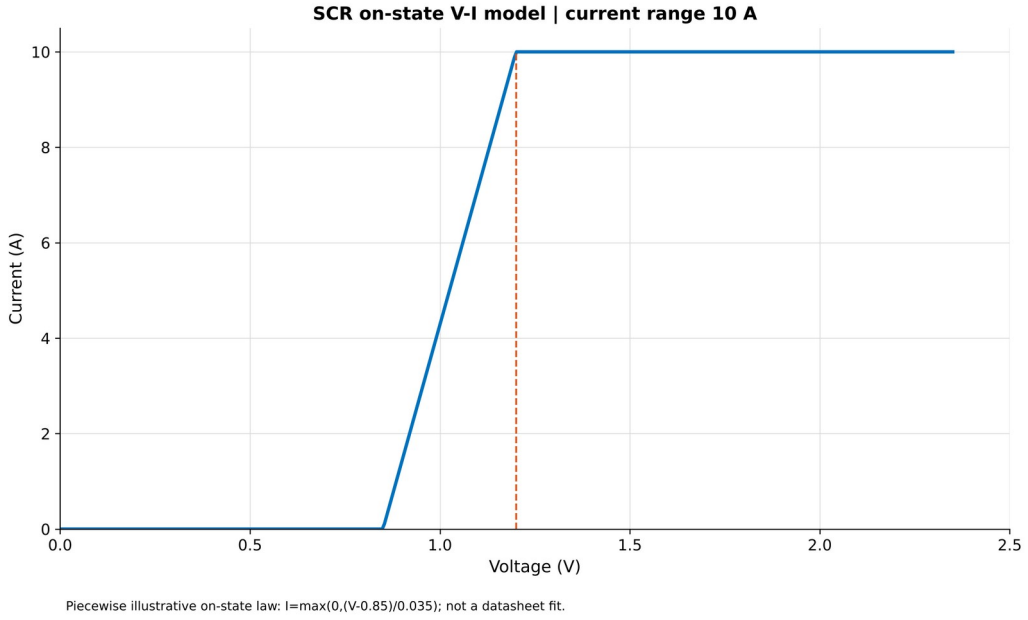
REFERENCE illustration SCR_002. Reproduce as SCR_002.png with the supplied MATLAB script.

Figure SCR_003 — SCR on-state V–I



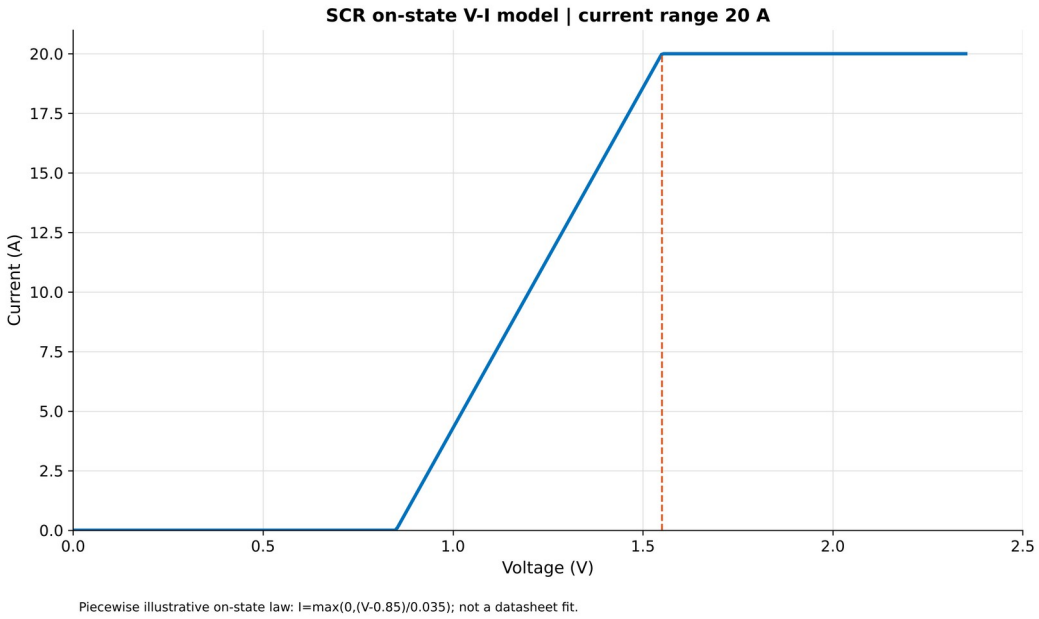
REFERENCE illustration SCR_003. Reproduce as SCR_003.png with the supplied MATLAB script.

Figure SCR_004 — SCR on-state V–I



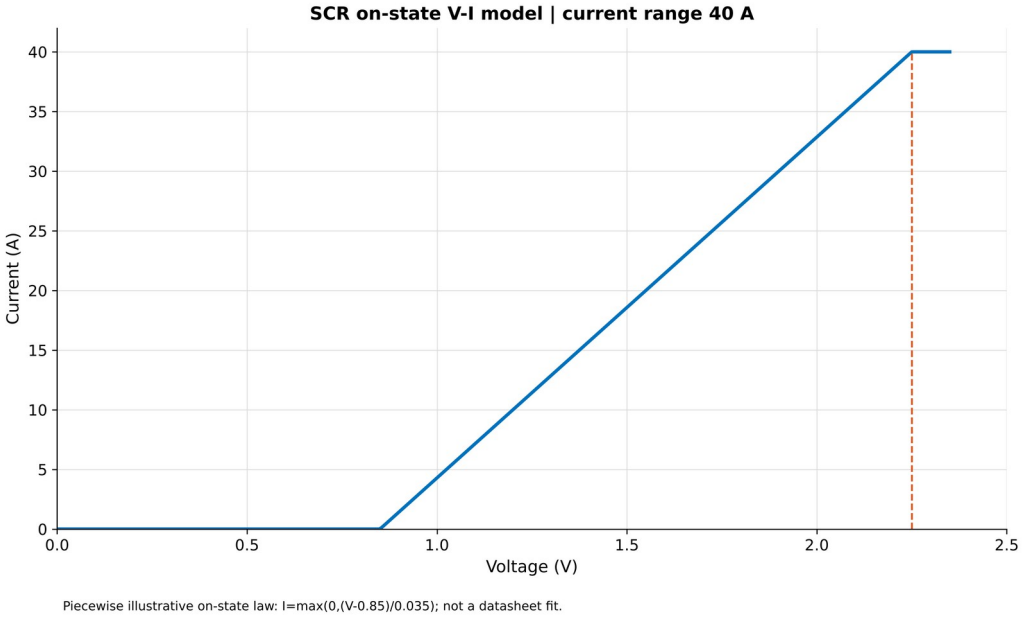
REFERENCE illustration SCR_004. Reproduce as SCR_004.png with the supplied MATLAB script.

Figure SCR_005 — SCR on-state V–I



REFERENCE illustration SCR_005. Reproduce as SCR_005.png with the supplied MATLAB script.

Figure SCR_006 — SCR on-state V–I

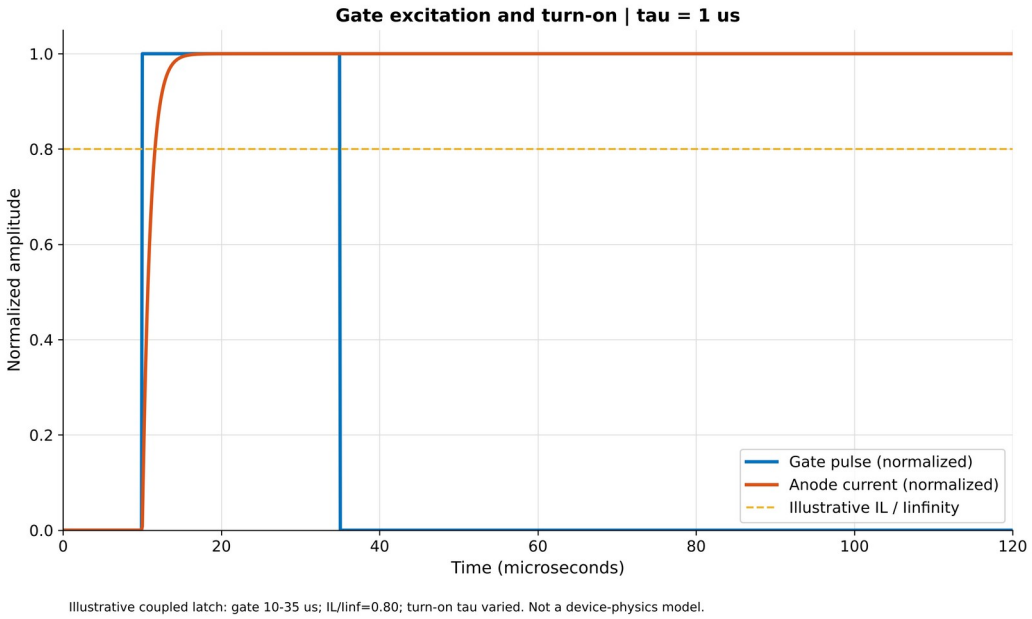


REFERENCE illustration SCR_006. Reproduce as SCR_006.png with the supplied MATLAB script.

007–013 | Gate excitation and latching

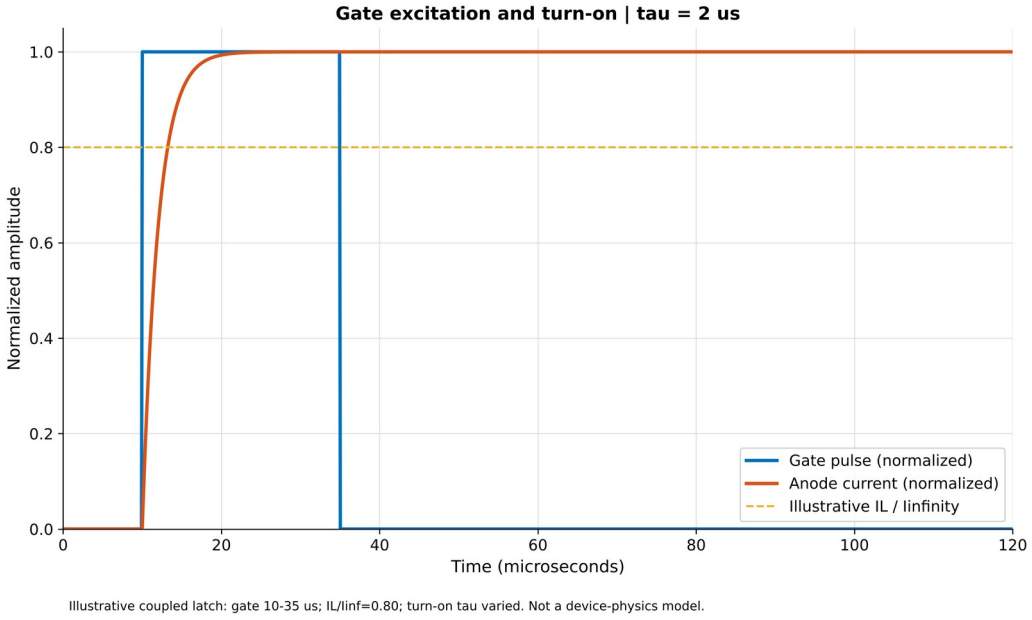
Rectangular gate pulse, normalized anode-current rise and a chosen latching threshold.

Figure SCR_007 — Gate excitation and latching



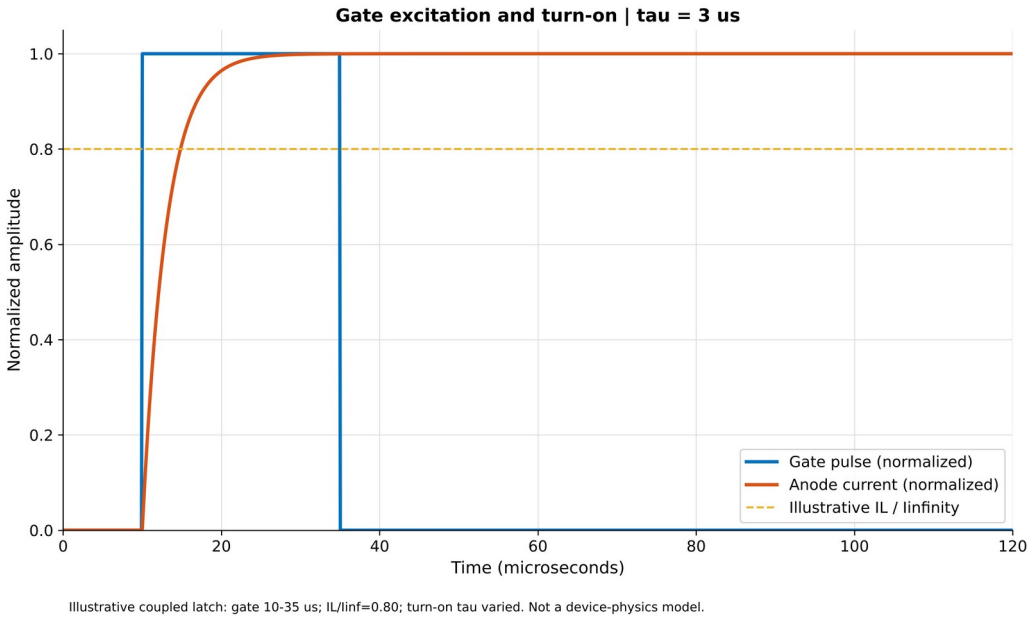
REFERENCE illustration SCR_007. Reproduce as SCR_007.png with the supplied MATLAB script.

Figure SCR_008 — Gate excitation and latching



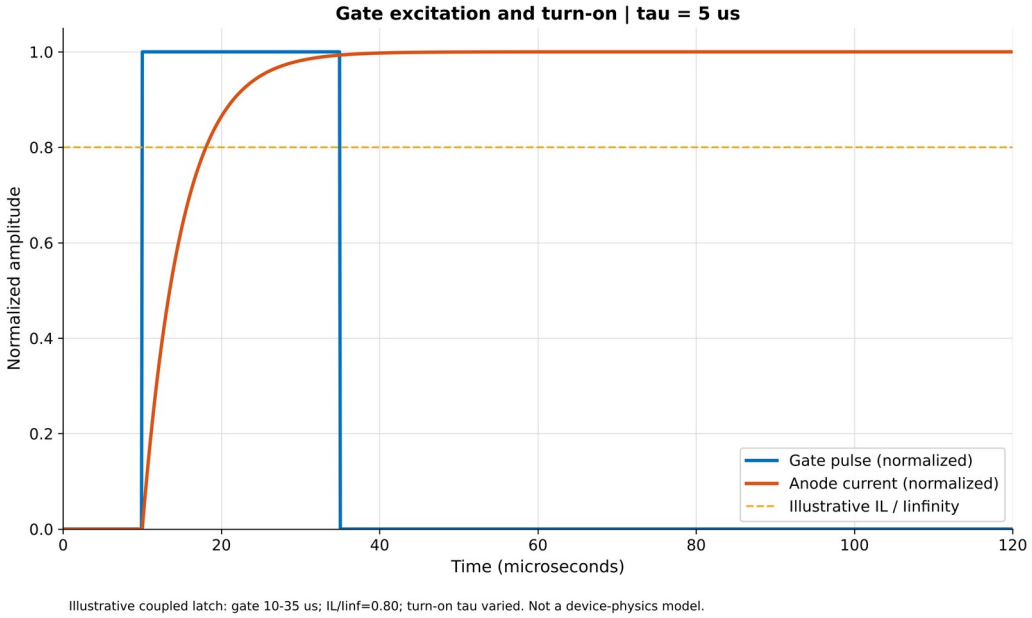
REFERENCE illustration SCR_008. Reproduce as SCR_008.png with the supplied MATLAB script.

Figure SCR_009 — Gate excitation and latching



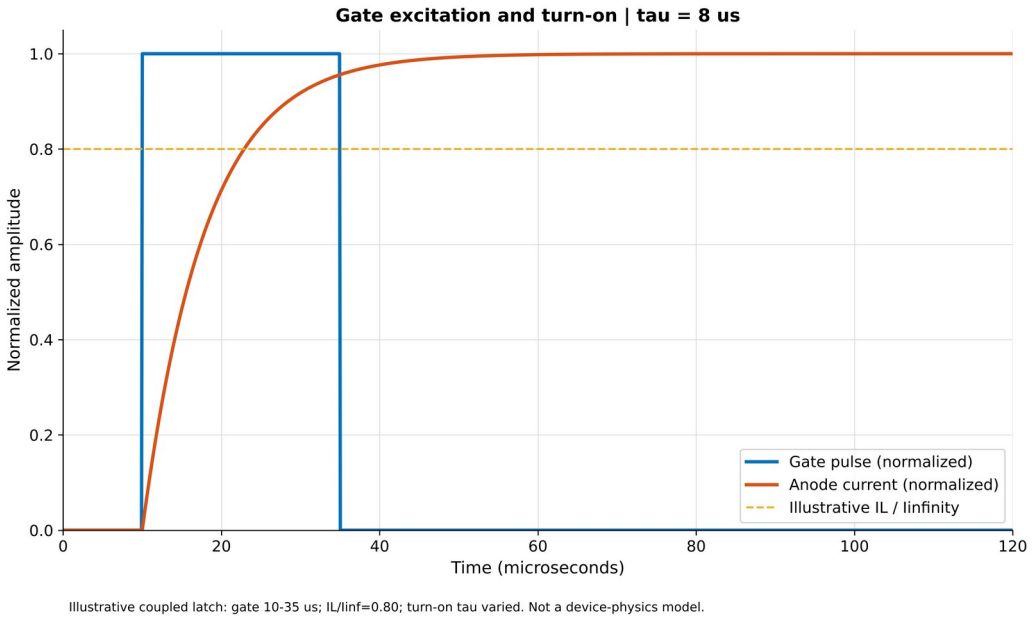
REFERENCE illustration SCR_009. Reproduce as SCR_009.png with the supplied MATLAB script.

Figure SCR_010 — Gate excitation and latching



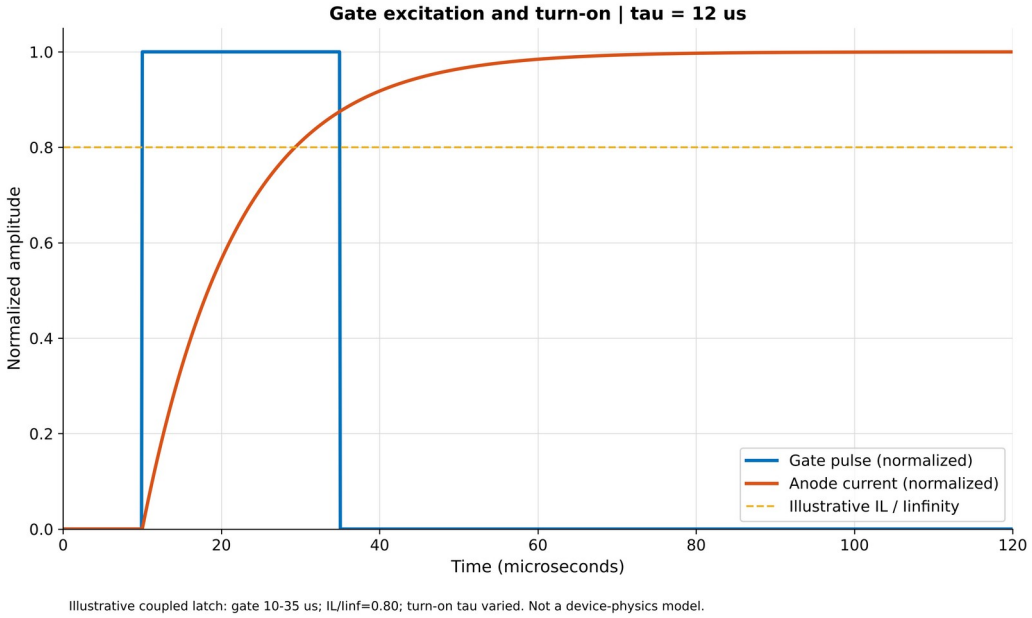
REFERENCE illustration SCR_010. Reproduce as SCR_010.png with the supplied MATLAB script.

Figure SCR_011 — Gate excitation and latching



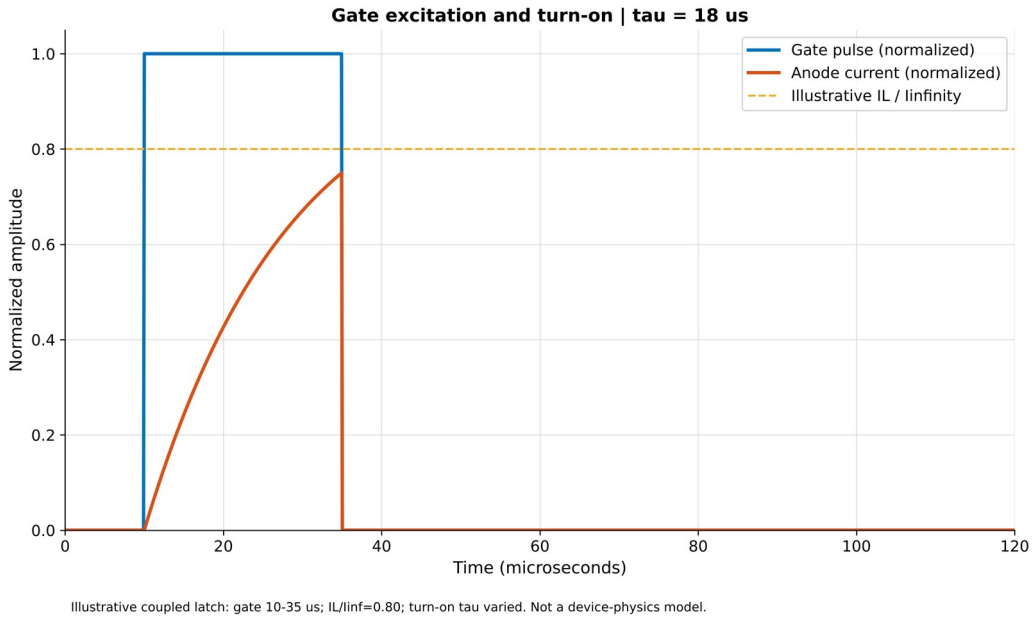
REFERENCE illustration SCR_011. Reproduce as SCR_011.png with the supplied MATLAB script.

Figure SCR_012 — Gate excitation and latching



REFERENCE illustration SCR_012. Reproduce as SCR_012.png with the supplied MATLAB script.

Figure SCR_013 — Gate excitation and latching

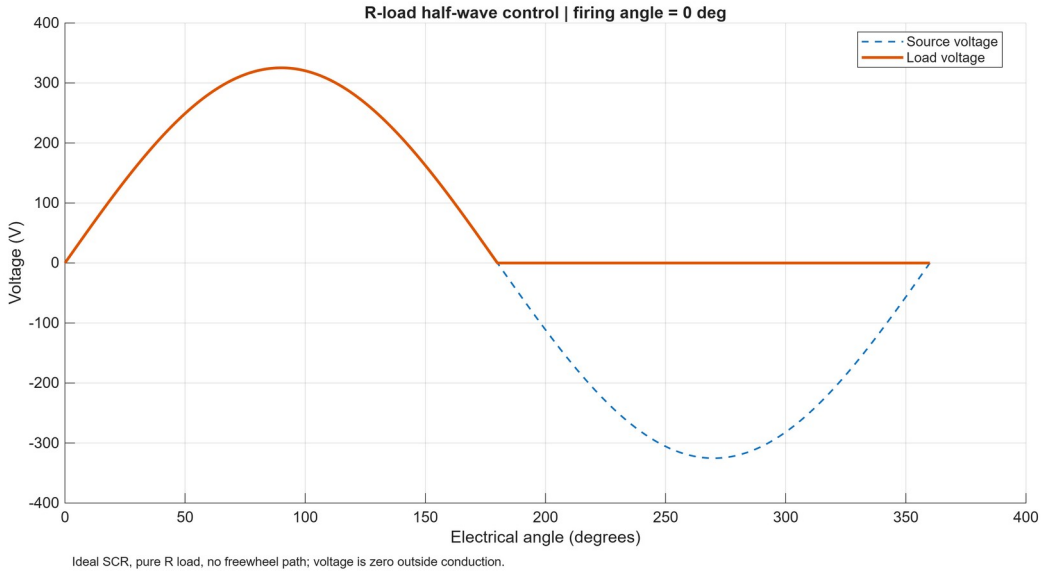


REFERENCE illustration SCR_013. Reproduce as SCR_013.png with the supplied MATLAB script.

014–021 | Phase-controlled resistive load

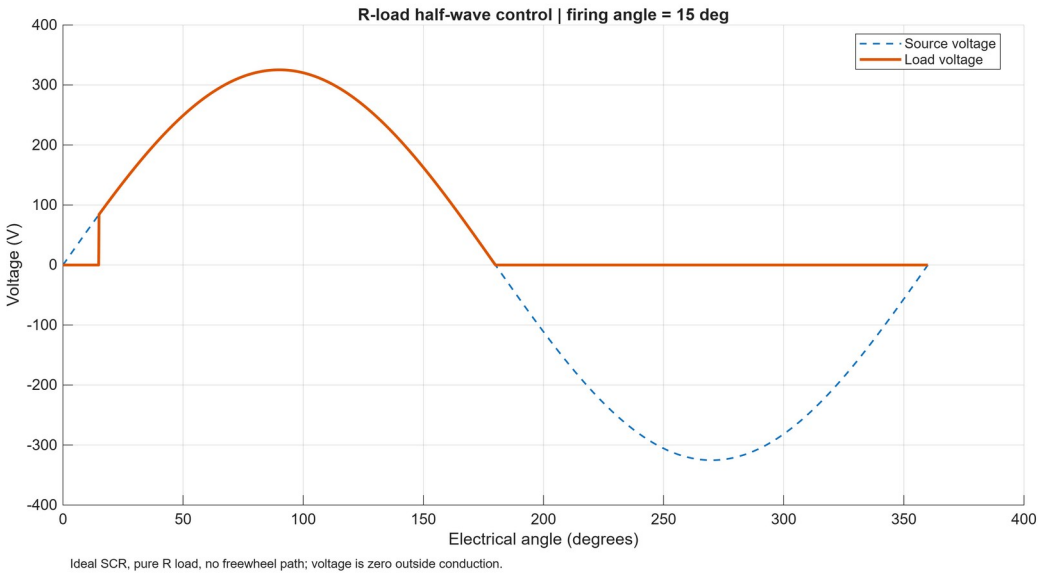
Ideal half-wave rectifier for different firing angles with pure resistive load.

Figure SCR_014 — Phase-controlled resistive load



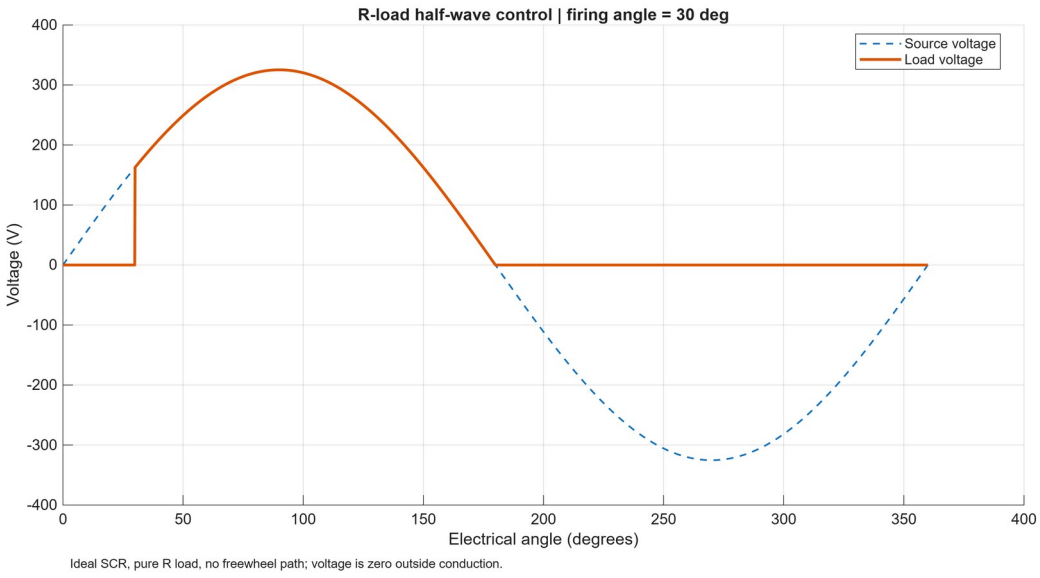
REFERENCE illustration SCR_014. Reproduce as SCR_014.png with the supplied MATLAB script.

Figure SCR_015 — Phase-controlled resistive load



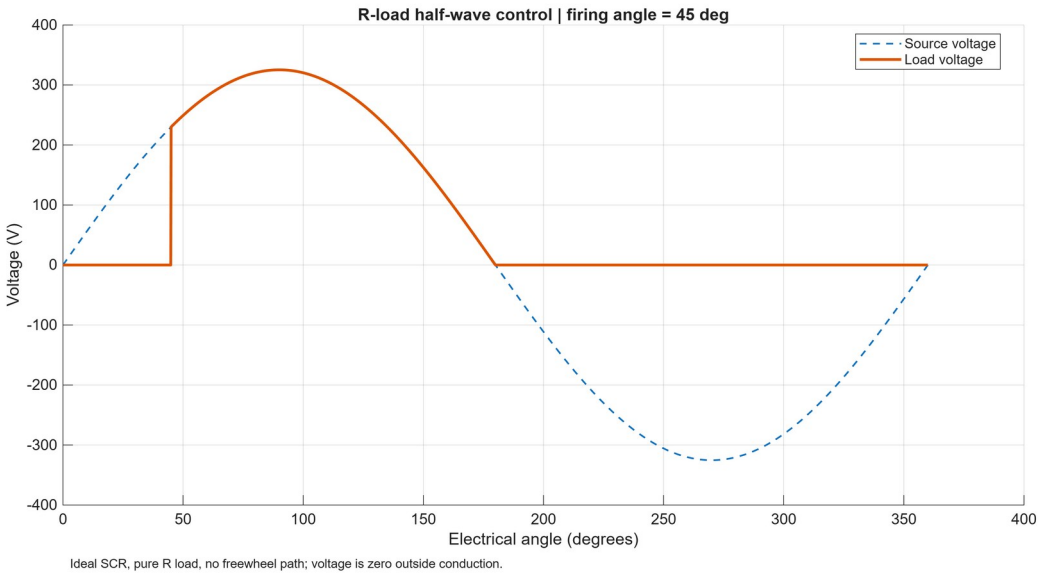
REFERENCE illustration SCR_015. Reproduce as SCR_015.png with the supplied MATLAB script.

Figure SCR_016 — Phase-controlled resistive load



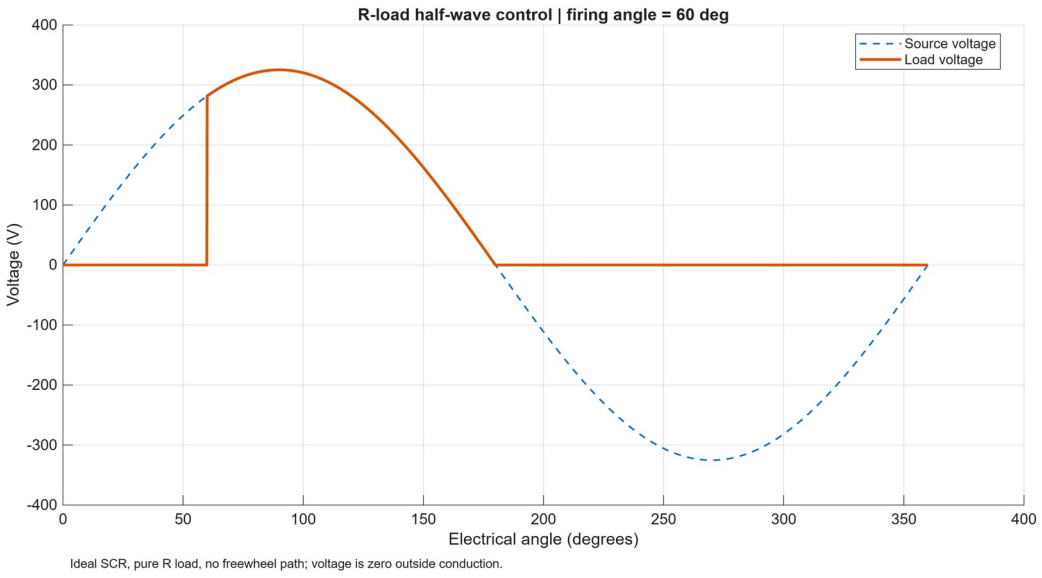
REFERENCE illustration SCR_016. Reproduce as SCR_016.png with the supplied MATLAB script.

Figure SCR_017 — Phase-controlled resistive load



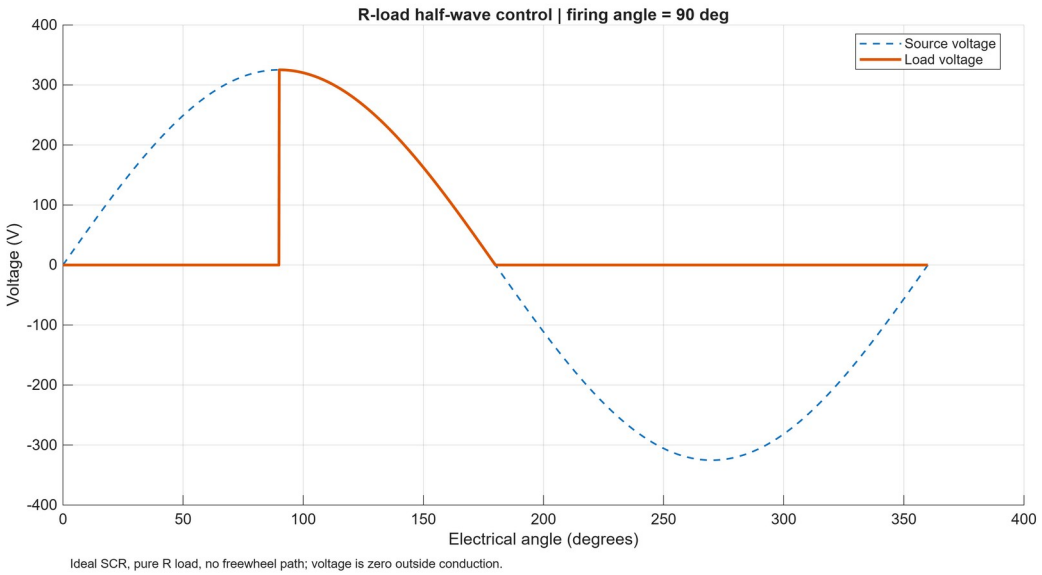
REFERENCE illustration SCR_017. Reproduce as SCR_017.png with the supplied MATLAB script.

Figure SCR_018 — Phase-controlled resistive load



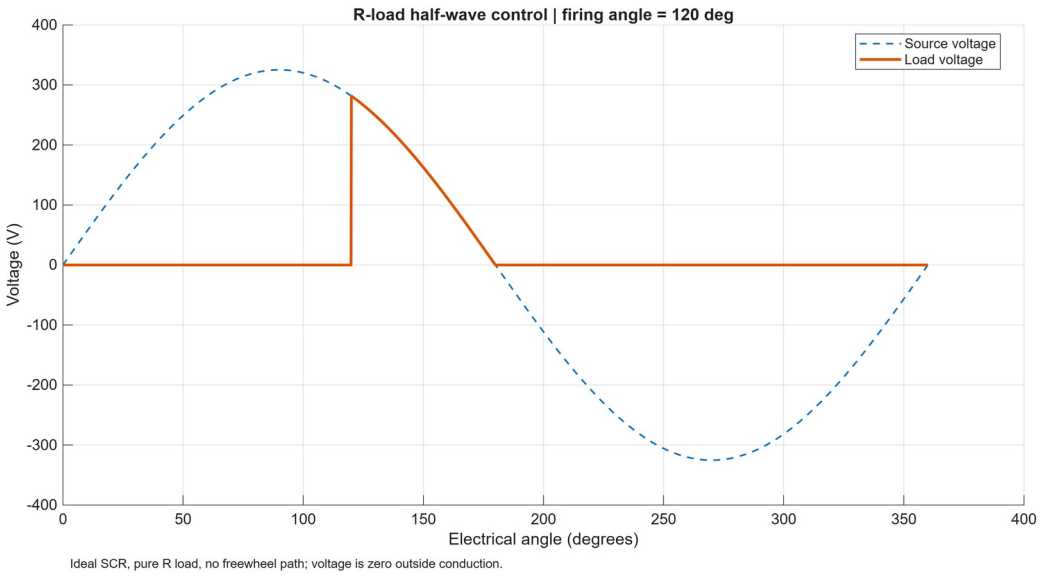
REFERENCE illustration SCR_018. Reproduce as SCR_018.png with the supplied MATLAB script.

Figure SCR_019 — Phase-controlled resistive load



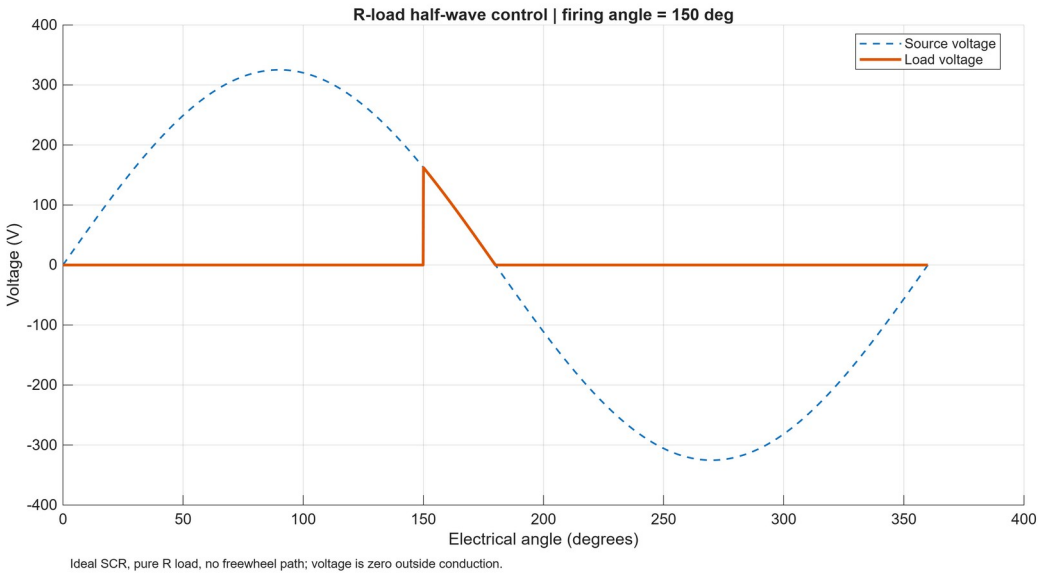
REFERENCE illustration SCR_019. Reproduce as SCR_019.png with the supplied MATLAB script.

Figure SCR_020 — Phase-controlled resistive load



REFERENCE illustration SCR_020. Reproduce as SCR_020.png with the supplied MATLAB script.

Figure SCR_021 — Phase-controlled resistive load

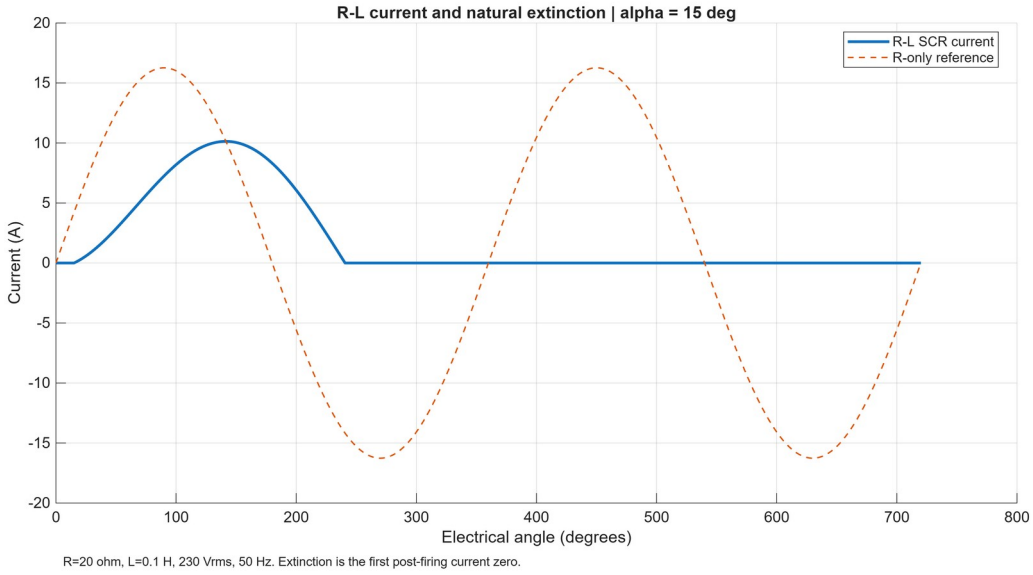


REFERENCE illustration SCR_021. Reproduce as SCR_021.png with the supplied MATLAB script.

022–031 | Inductive R–L load

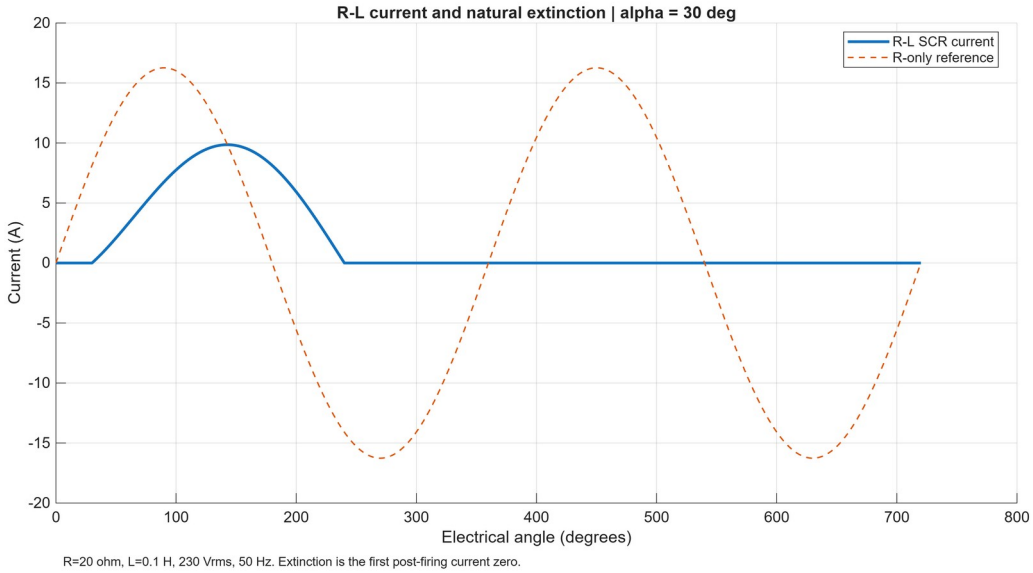
Analytical SCR current and first natural extinction after a firing event.

Figure SCR_022 — Inductive R–L load



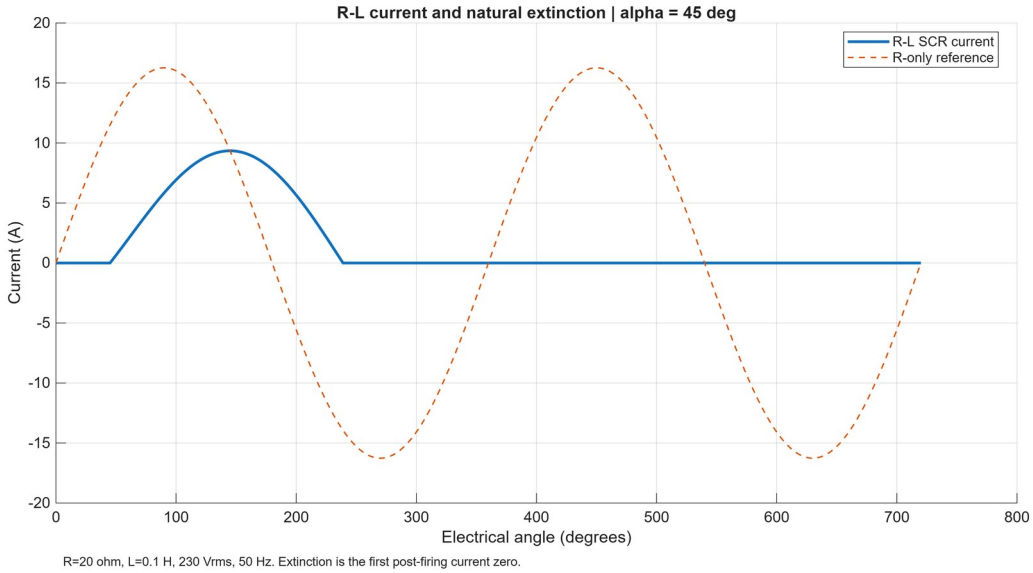
REFERENCE illustration SCR_022. Reproduce as SCR_022.png with the supplied MATLAB script.

Figure SCR_023 — Inductive R–L load



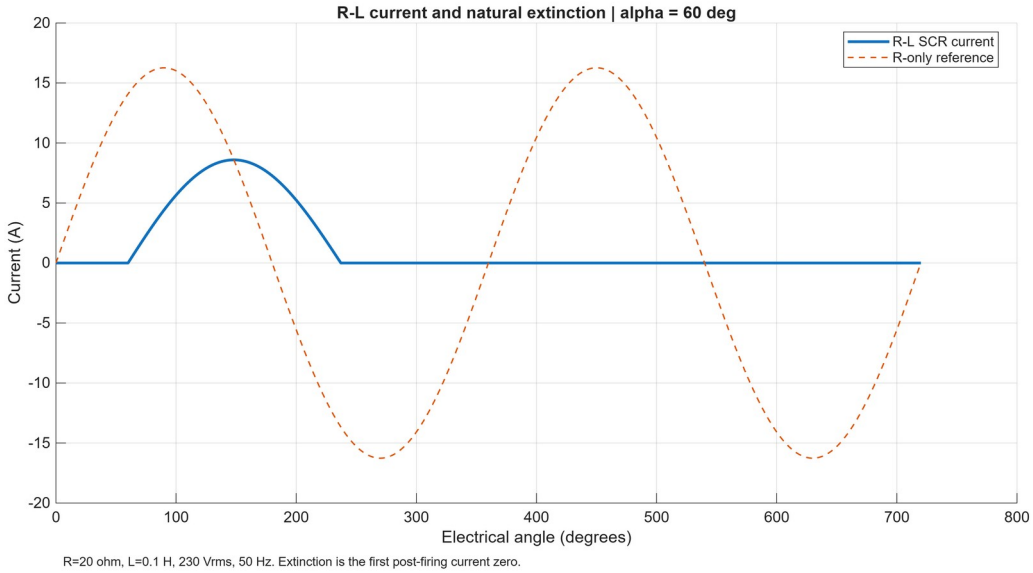
REFERENCE illustration SCR_023. Reproduce as SCR_023.png with the supplied MATLAB script.

Figure SCR_024 — Inductive R–L load



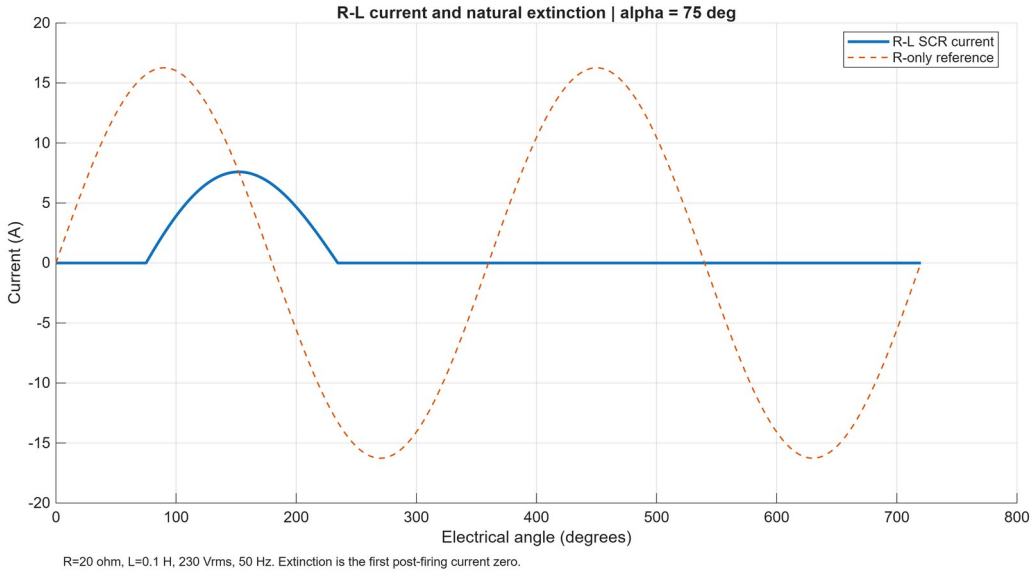
REFERENCE illustration SCR_024. Reproduce as SCR_024.png with the supplied MATLAB script.

Figure SCR_025 — Inductive R–L load



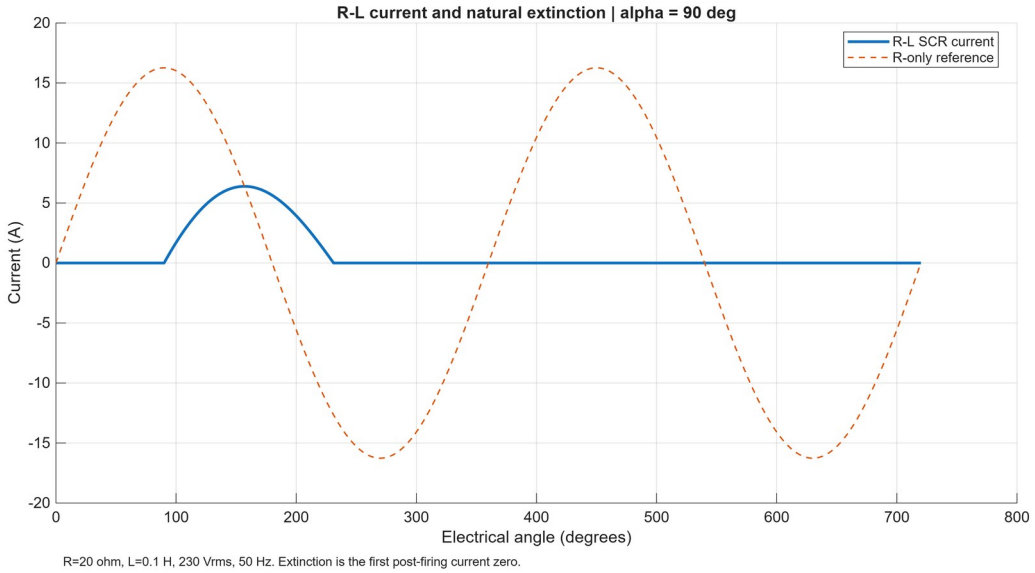
REFERENCE illustration SCR_025. Reproduce as SCR_025.png with the supplied MATLAB script.

Figure SCR_026 — Inductive R–L load



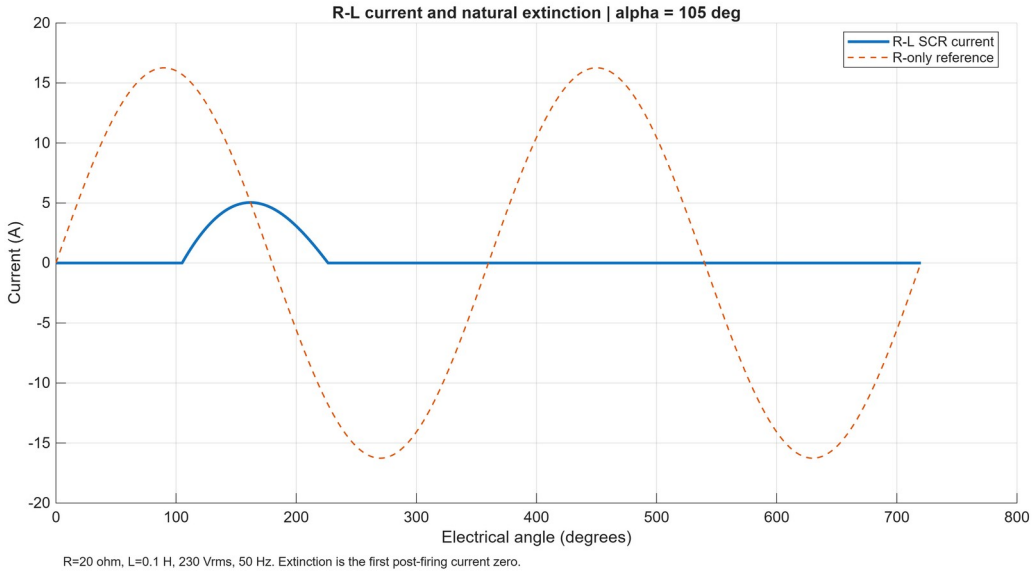
REFERENCE illustration SCR_026. Reproduce as SCR_026.png with the supplied MATLAB script.

Figure SCR_027 — Inductive R–L load



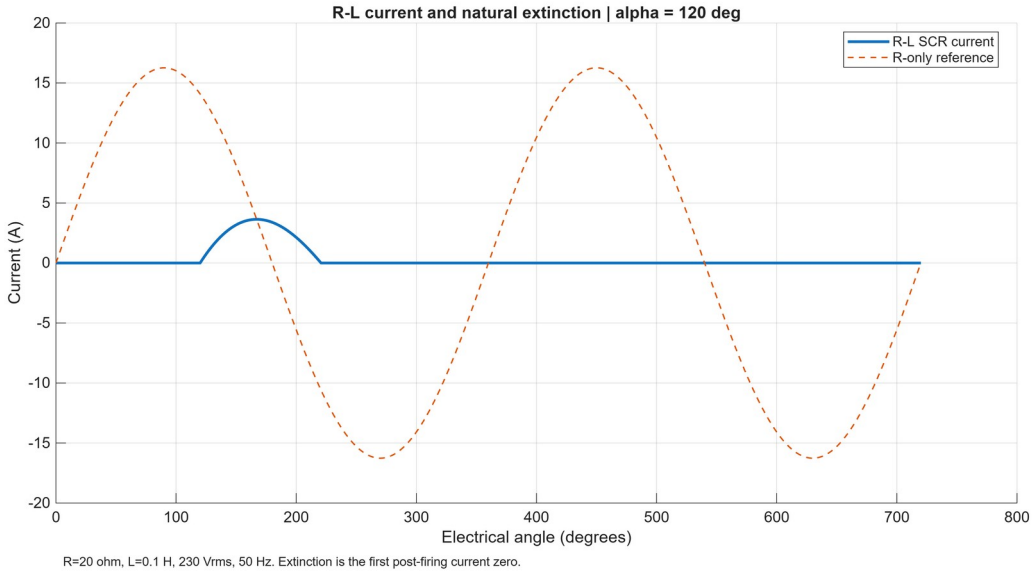
REFERENCE illustration SCR_027. Reproduce as SCR_027.png with the supplied MATLAB script.

Figure SCR_028 — Inductive R–L load



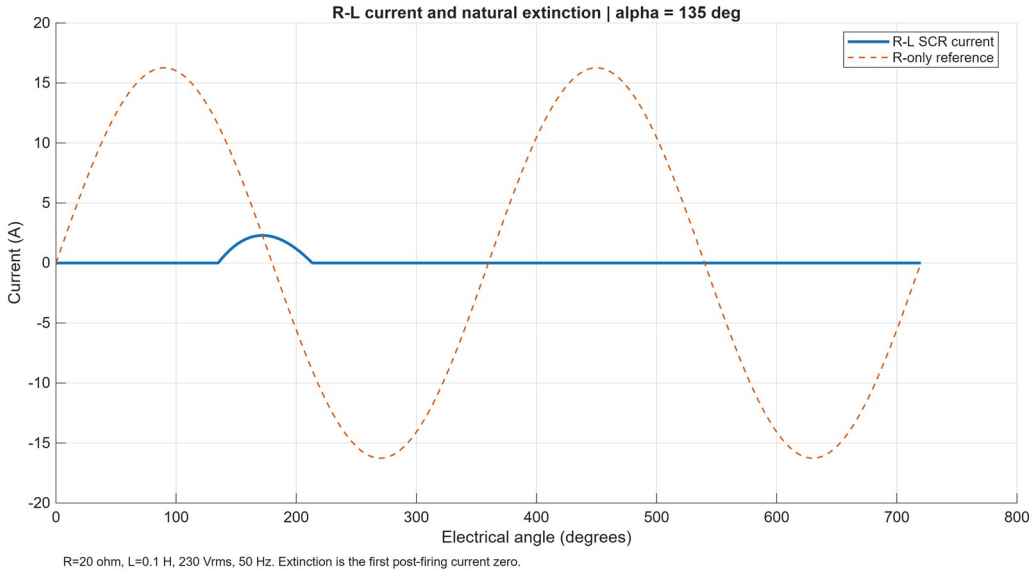
REFERENCE illustration SCR_028. Reproduce as SCR_028.png with the supplied MATLAB script.

Figure SCR_029 — Inductive R–L load



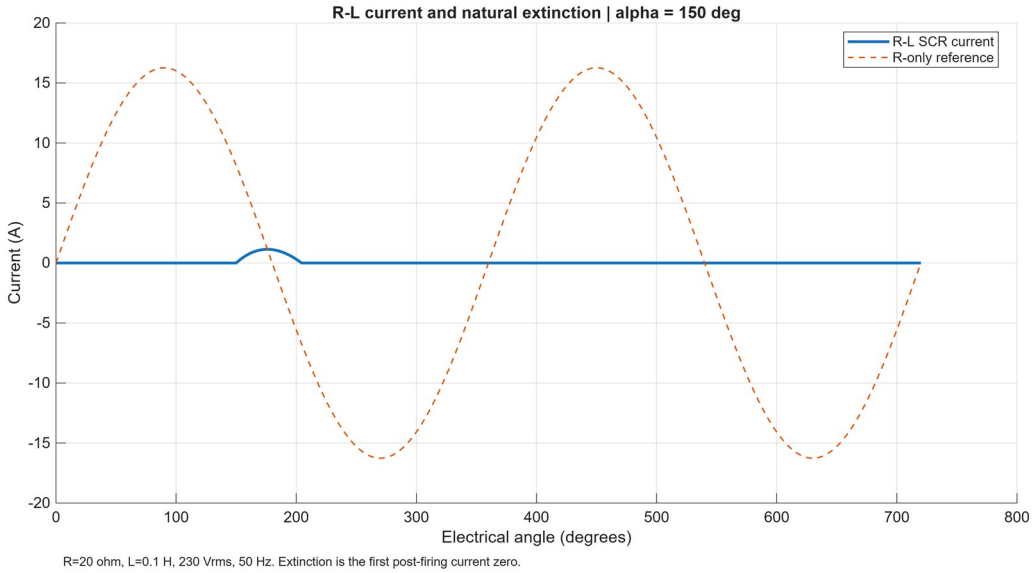
REFERENCE illustration SCR_029. Reproduce as SCR_029.png with the supplied MATLAB script.

Figure SCR_030 — Inductive R–L load



REFERENCE illustration SCR_030. Reproduce as SCR_030.png with the supplied MATLAB script.

Figure SCR_031 — Inductive R–L load

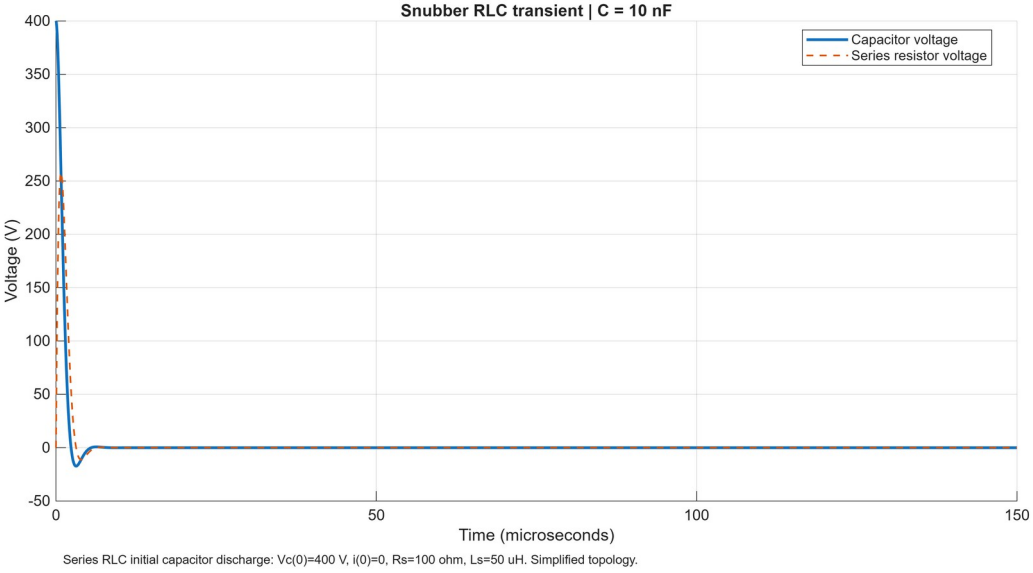


REFERENCE illustration SCR_031. Reproduce as SCR_031.png with the supplied MATLAB script.

032–041 | R–L–C transient

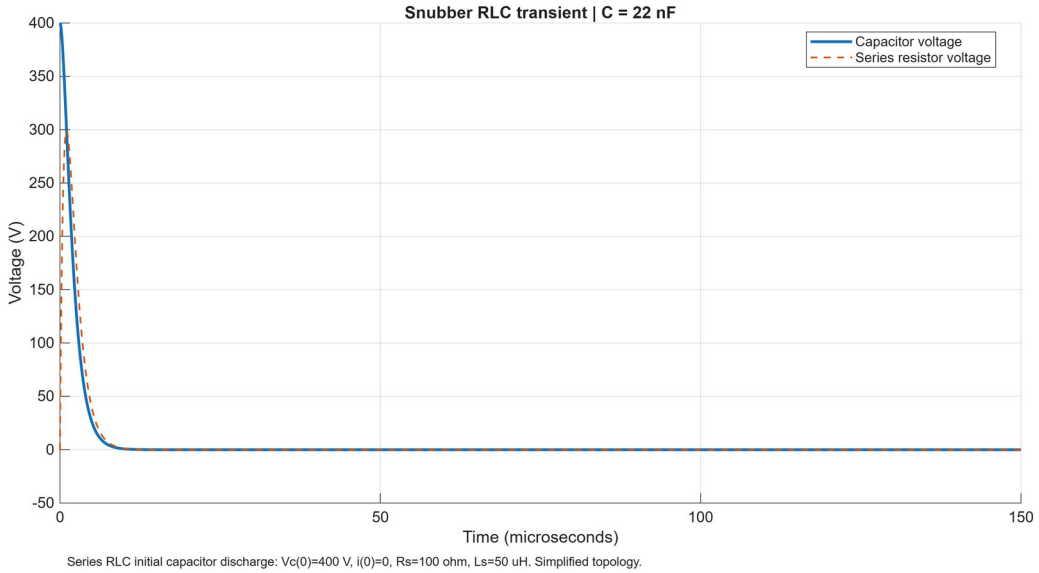
Series RLC initial capacitor discharge used as a simplified snubber proxy.

Figure SCR_032 — R–L–C transient



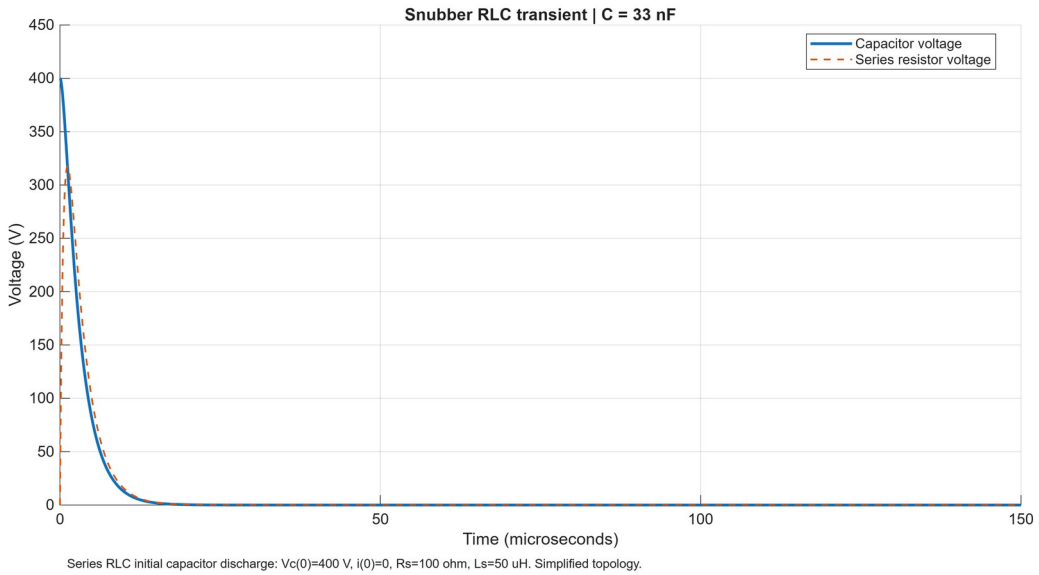
REFERENCE illustration SCR_032. Reproduce as SCR_032.png with the supplied MATLAB script.

Figure SCR_033 — R–L–C transient



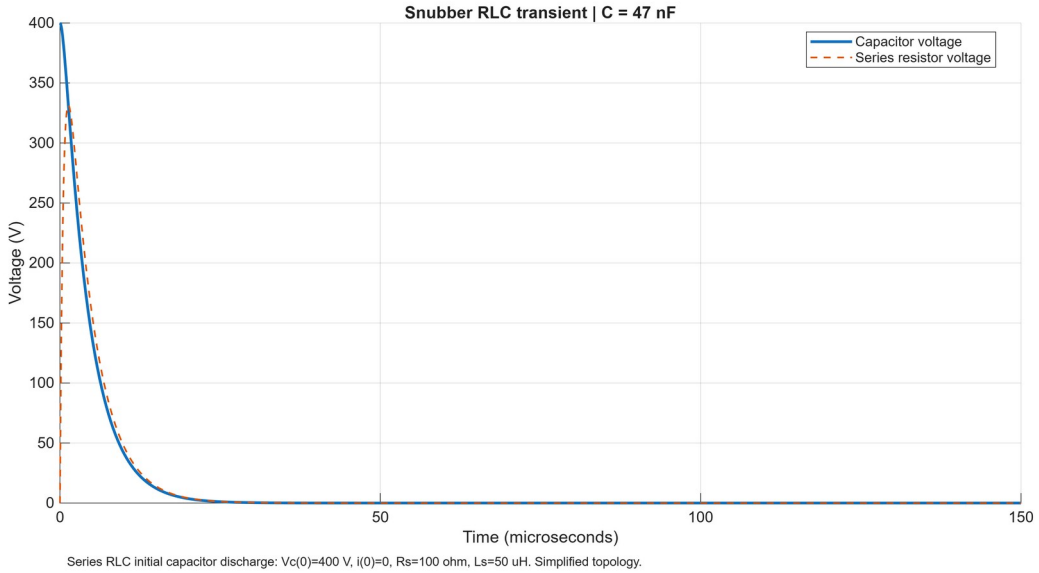
REFERENCE illustration SCR_033. Reproduce as SCR_033.png with the supplied MATLAB script.

Figure SCR_034 — R–L–C transient



REFERENCE illustration SCR_034. Reproduce as SCR_034.png with the supplied MATLAB script.

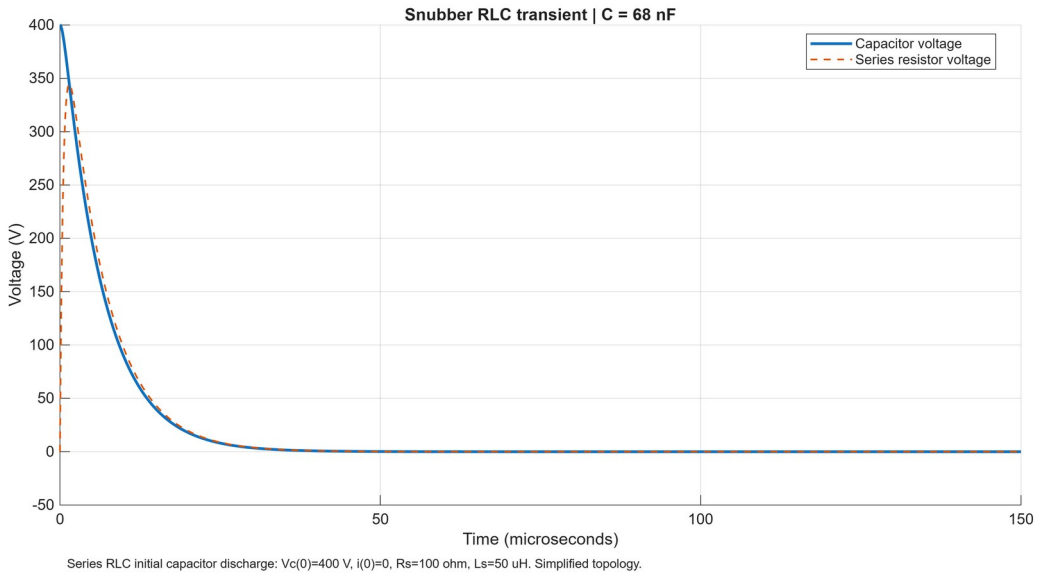
Figure SCR_035 — R–L–C transient



Series RLC initial capacitor discharge: Vc(0)=400 V, i(0)=0, Rs=100 ohm, Ls=50 uH. Simplified topology.

REFERENCE illustration SCR_035. Reproduce as SCR_035.png with the supplied MATLAB script.

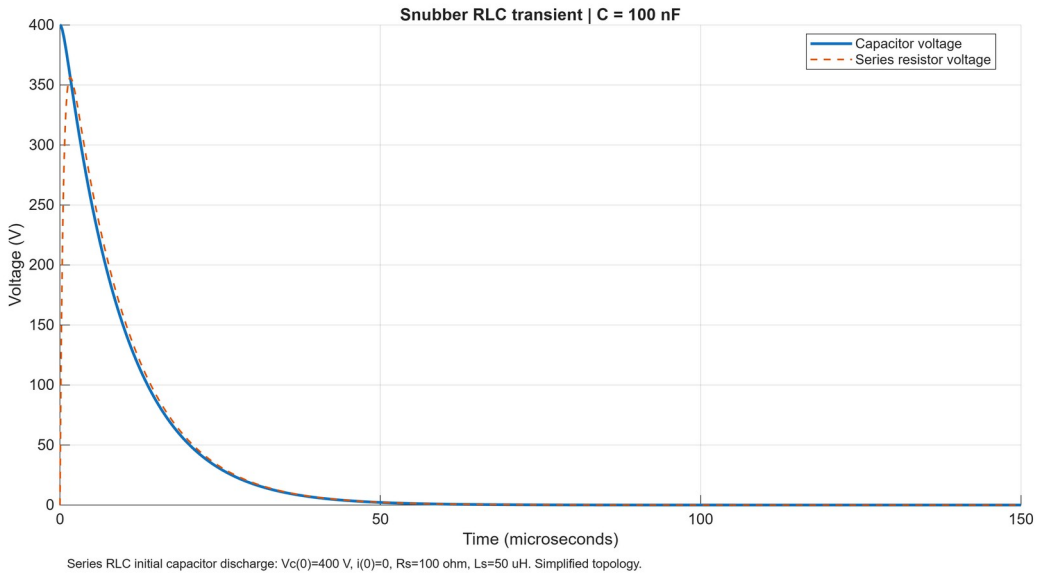
Figure SCR_036 — R–L–C transient



Series RLC initial capacitor discharge: Vc(0)=400 V, i(0)=0, Rs=100 ohm, Ls=50 uH. Simplified topology.

REFERENCE illustration SCR_036. Reproduce as SCR_036.png with the supplied MATLAB script.

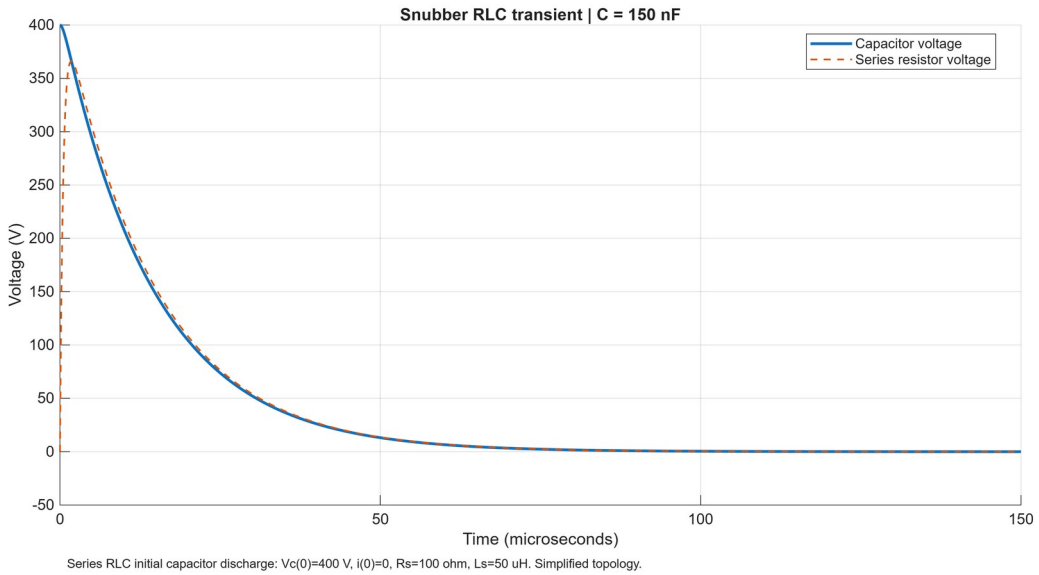
Figure SCR_037 — R–L–C transient



Series RLC initial capacitor discharge: Vc(0)=400 V, i(0)=0, Rs=100 ohm, Ls=50 uH. Simplified topology.

REFERENCE illustration SCR_037. Reproduce as SCR_037.png with the supplied MATLAB script.

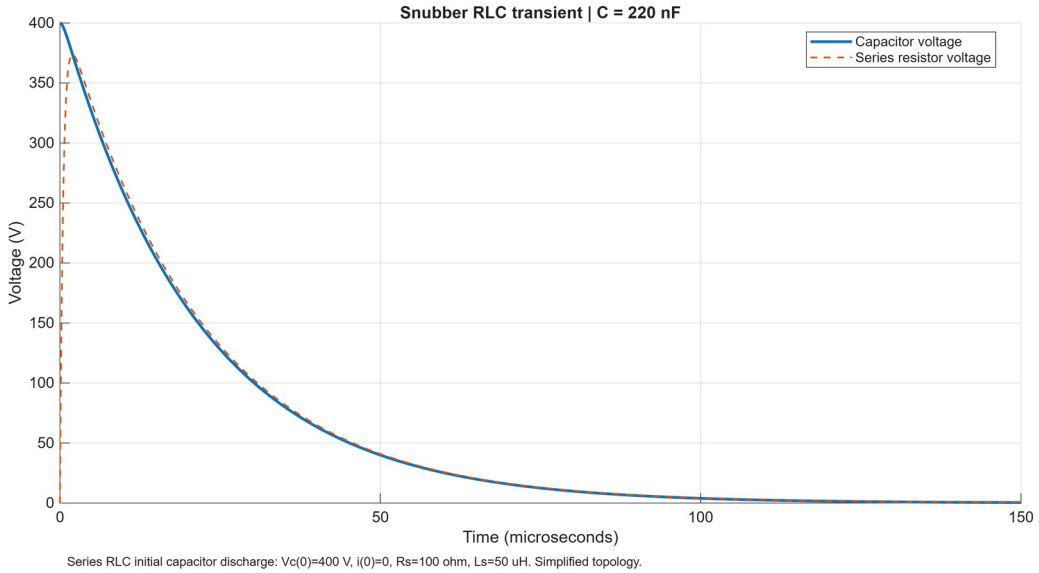
Figure SCR_038 — R–L–C transient



Series RLC initial capacitor discharge: Vc(0)=400 V, i(0)=0, Rs=100 ohm, Ls=50 uH. Simplified topology.

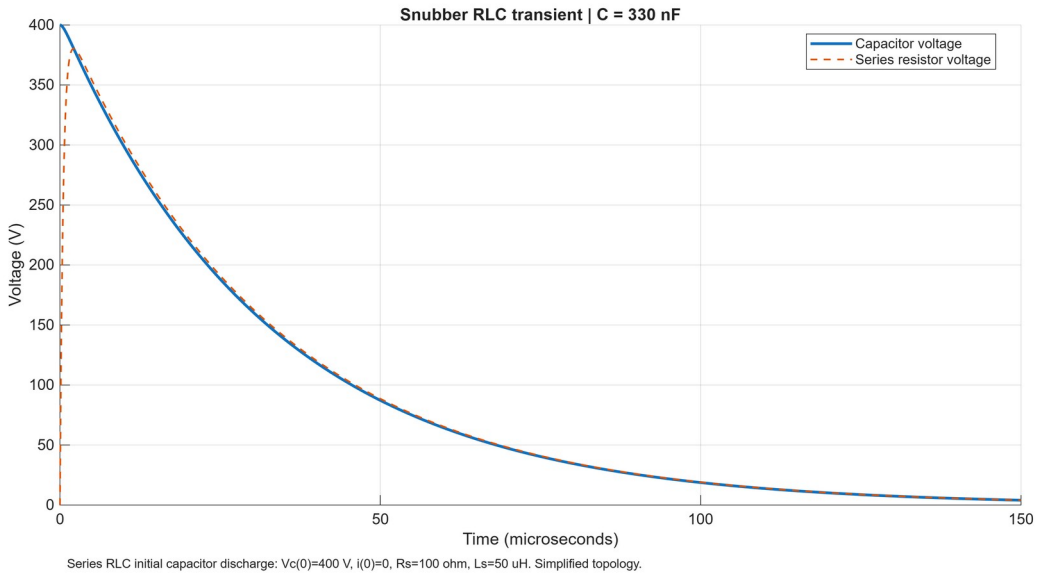
REFERENCE illustration SCR_038. Reproduce as SCR_038.png with the supplied MATLAB script.

Figure SCR_039 — R–L–C transient



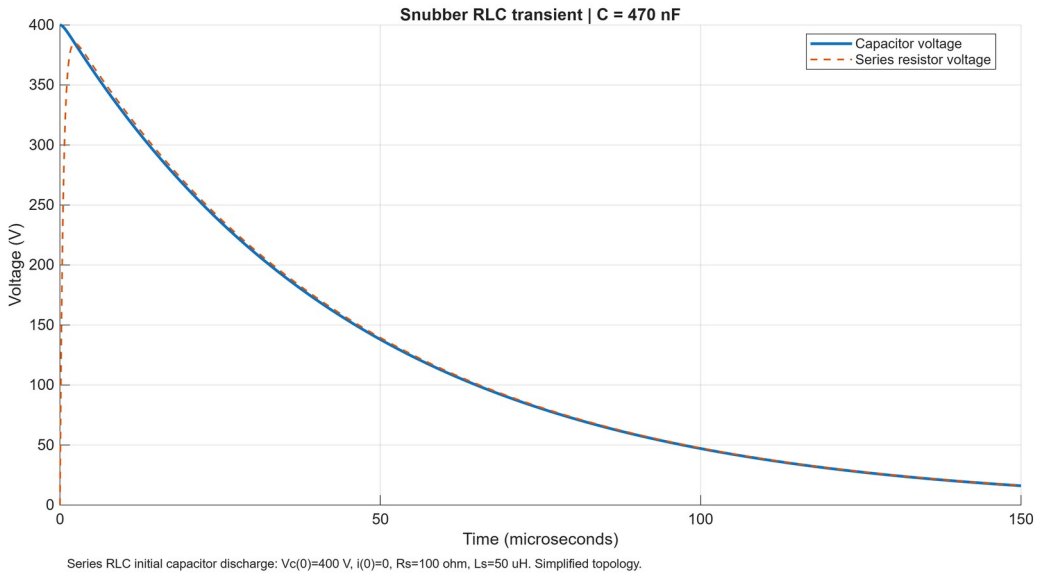
REFERENCE illustration SCR_039. Reproduce as SCR_039.png with the supplied MATLAB script.

Figure SCR_040 — R–L–C transient



REFERENCE illustration SCR_040. Reproduce as SCR_040.png with the supplied MATLAB script.

Figure SCR_041 — R–L–C transient

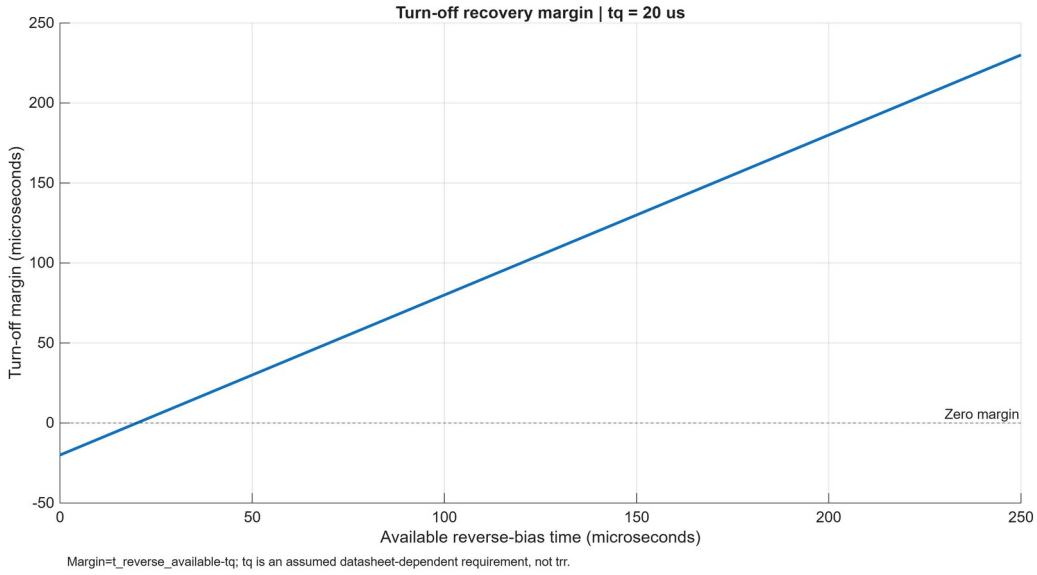


REFERENCE illustration SCR_041. Reproduce as SCR_041.png with the supplied MATLAB script.

042–048 | Turn-off margin

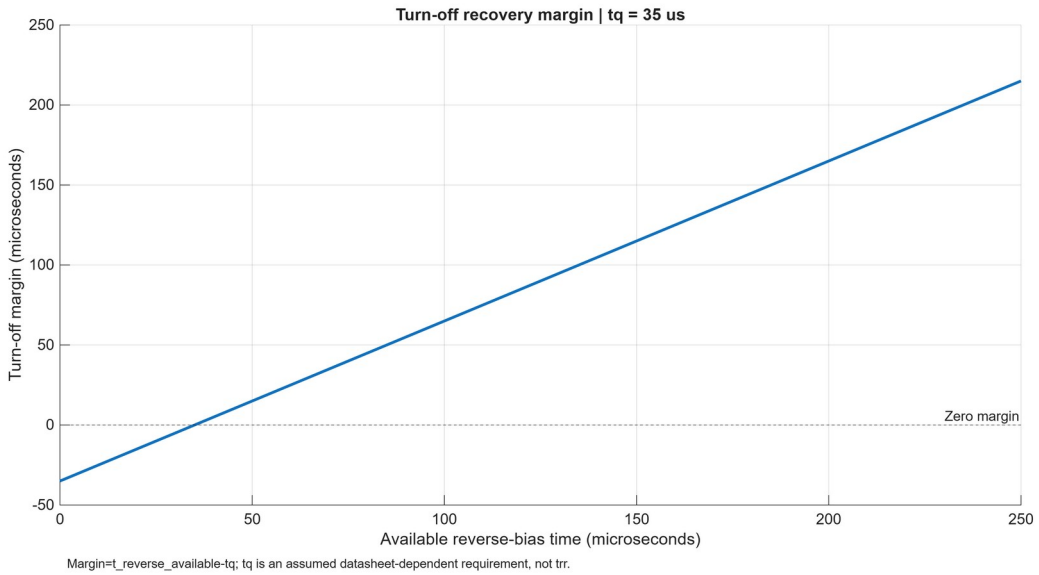
Available reverse-bias duration compared with assumed turn-off time tq.

Figure SCR_042 — Turn-off margin



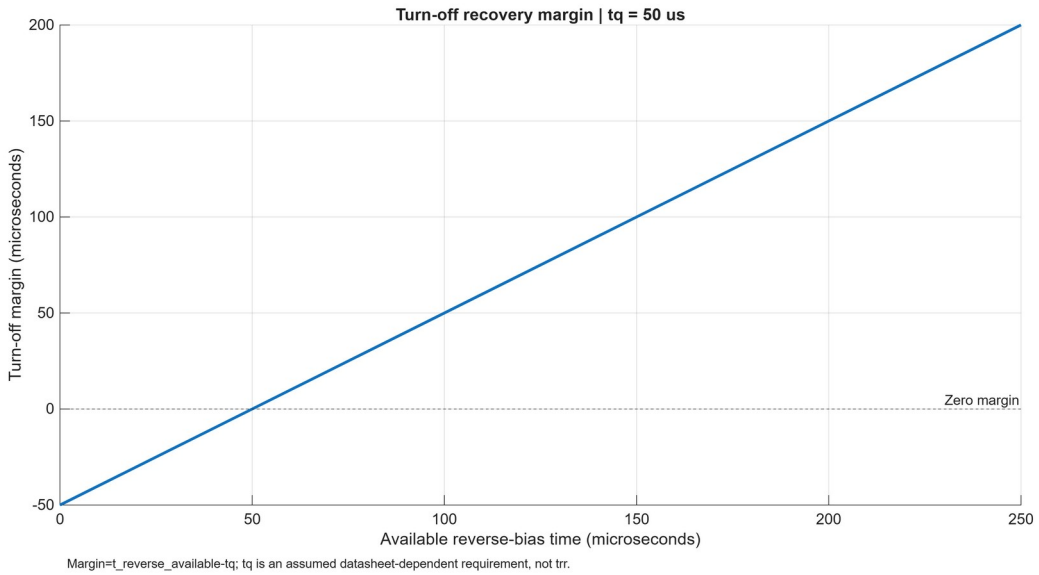
REFERENCE illustration SCR_042. Reproduce as SCR_042.png with the supplied MATLAB script.

Figure SCR_043 — Turn-off margin



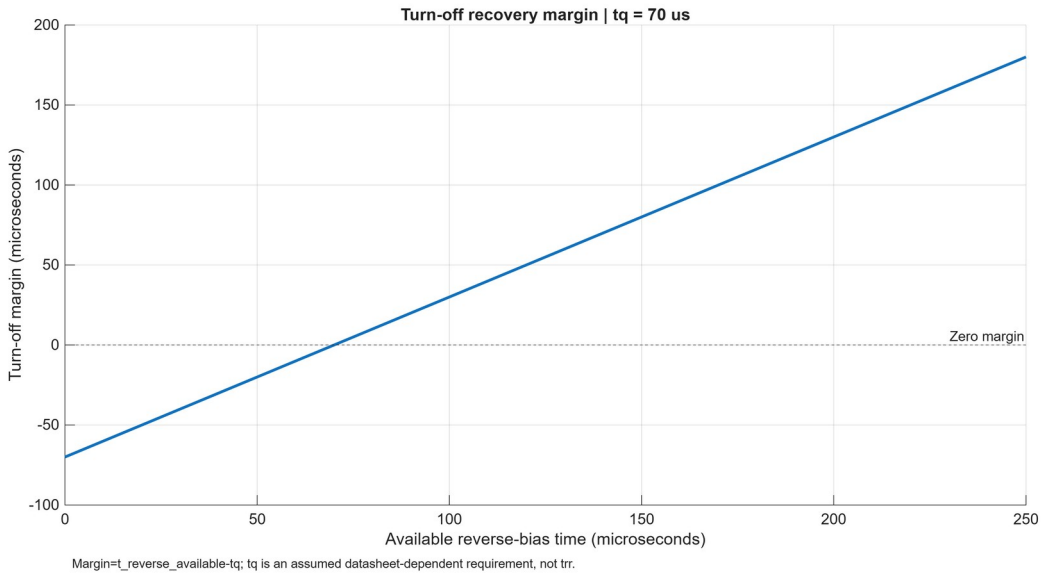
REFERENCE illustration SCR_043. Reproduce as SCR_043.png with the supplied MATLAB script.

Figure SCR_044 — Turn-off margin



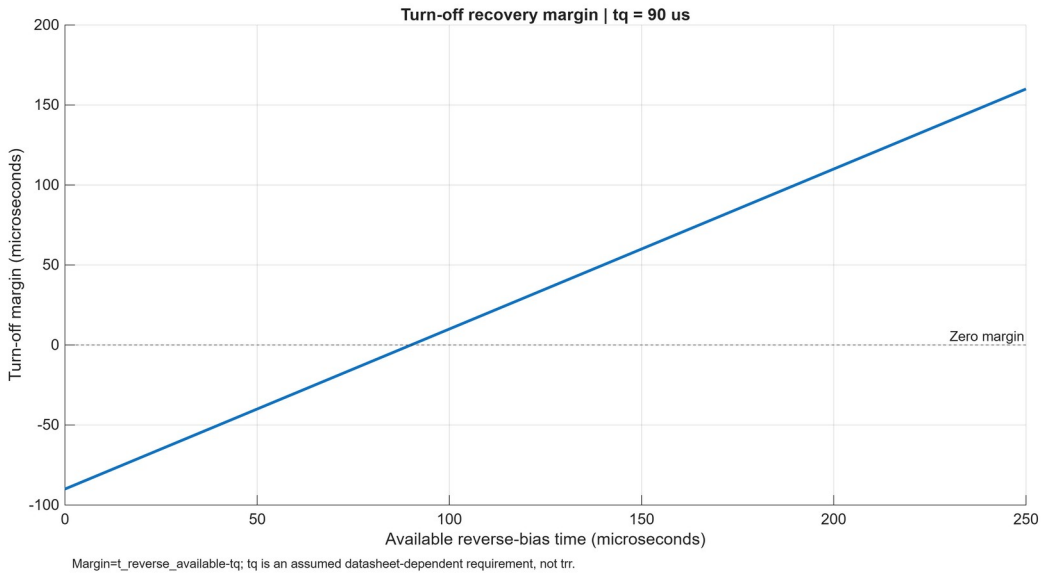
REFERENCE illustration SCR_044. Reproduce as SCR_044.png with the supplied MATLAB script.

Figure SCR_045 — Turn-off margin



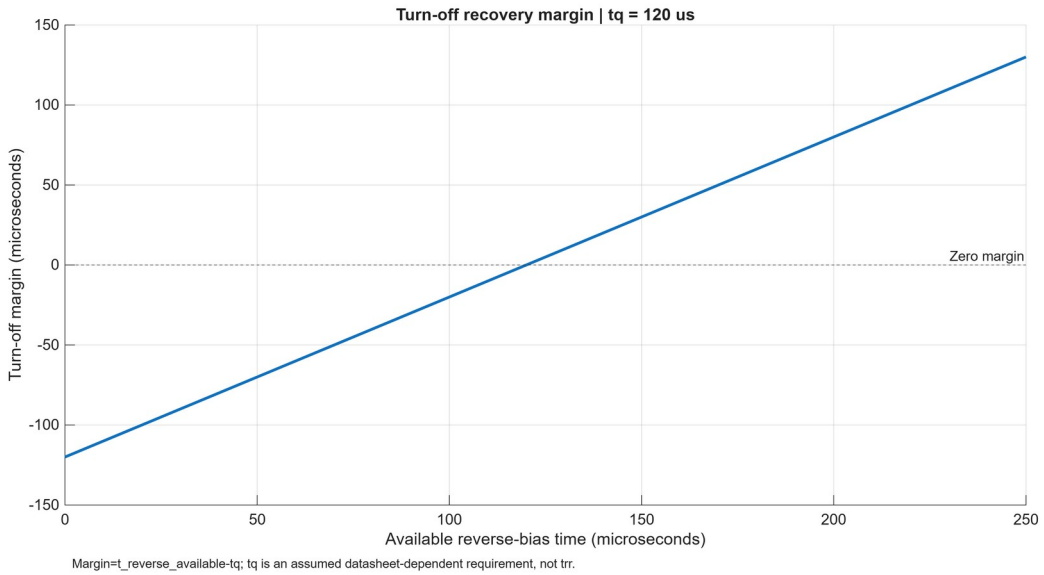
REFERENCE illustration SCR_045. Reproduce as SCR_045.png with the supplied MATLAB script.

Figure SCR_046 — Turn-off margin



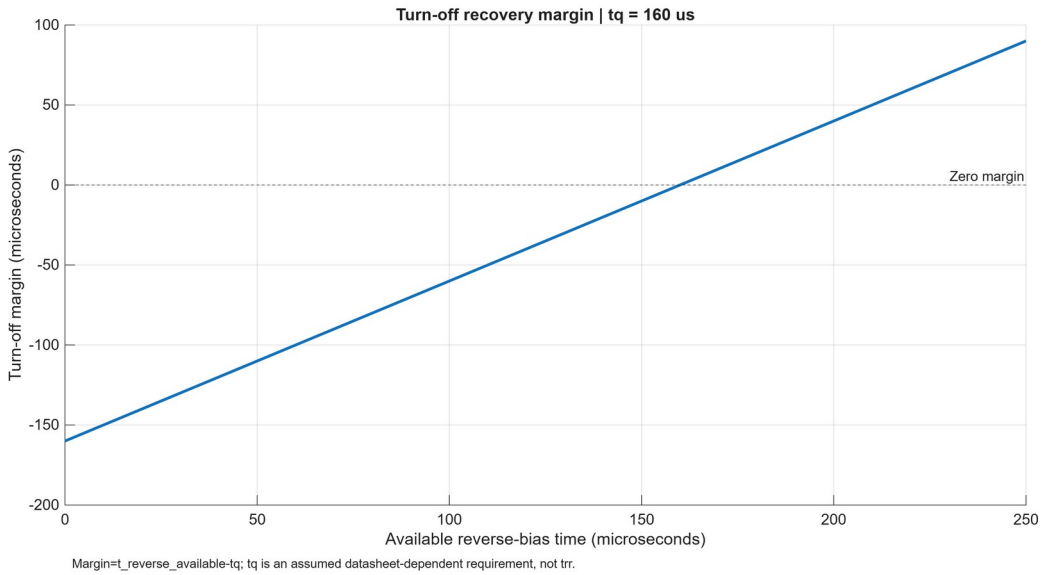
REFERENCE illustration SCR_046. Reproduce as SCR_046.png with the supplied MATLAB script.

Figure SCR_047 — Turn-off margin



REFERENCE illustration SCR_047. Reproduce as SCR_047.png with the supplied MATLAB script.

Figure SCR_048 — Turn-off margin

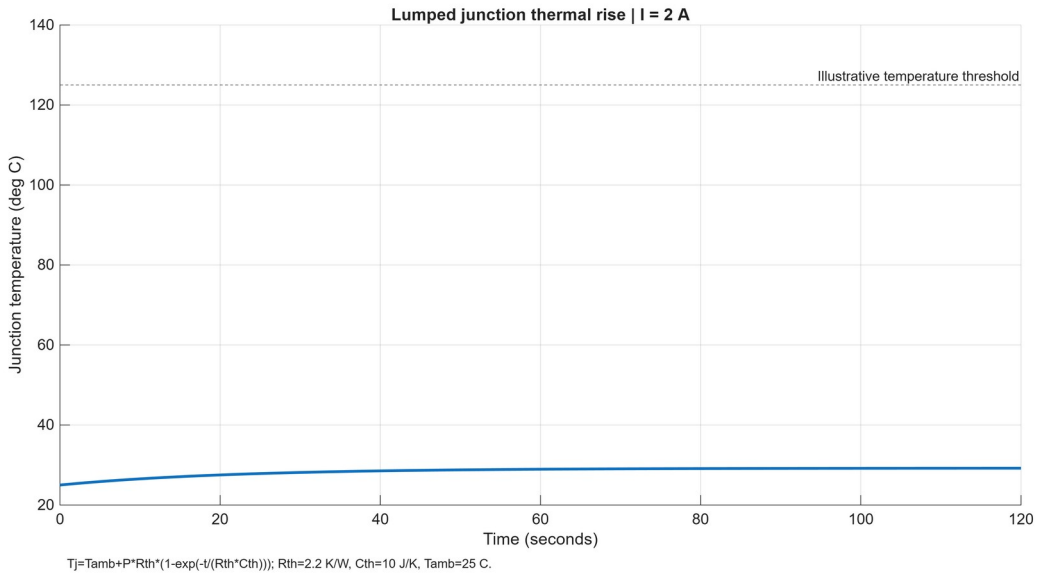


REFERENCE illustration SCR_048. Reproduce as SCR_048.png with the supplied MATLAB script.

049–055 | Lumped junction thermal response

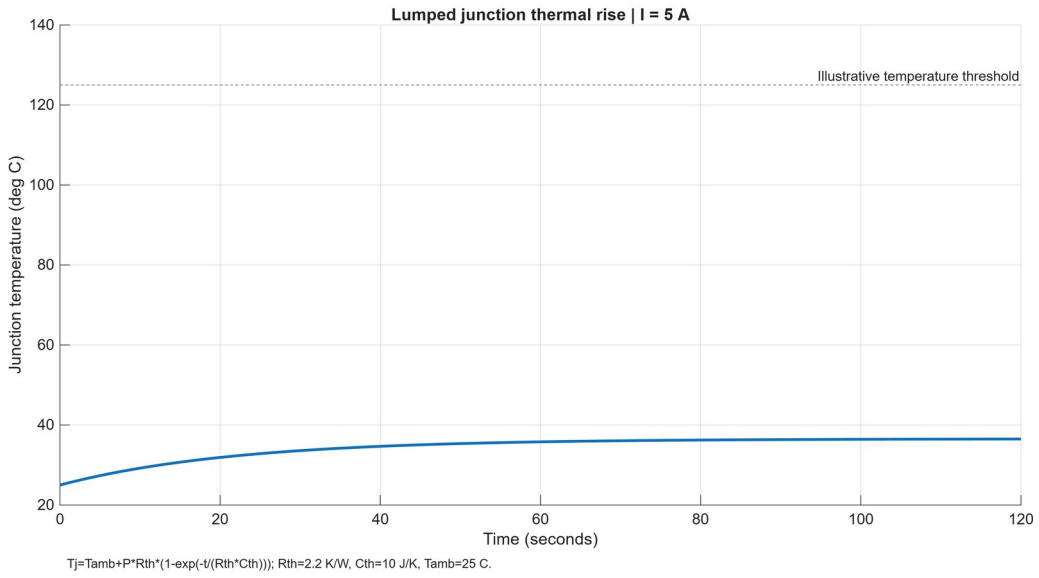
First-order thermal resistance/capacitance model and illustrative power loss.

Figure SCR_049 — Lumped junction thermal response



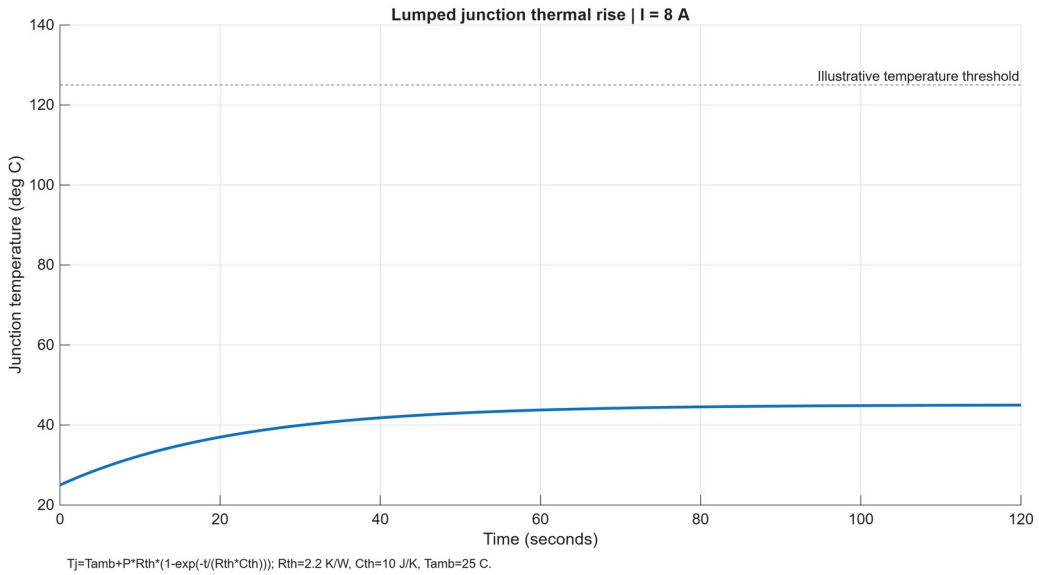
REFERENCE illustration SCR_049. Reproduce as SCR_049.png with the supplied MATLAB script.

Figure SCR_050 — Lumped junction thermal response



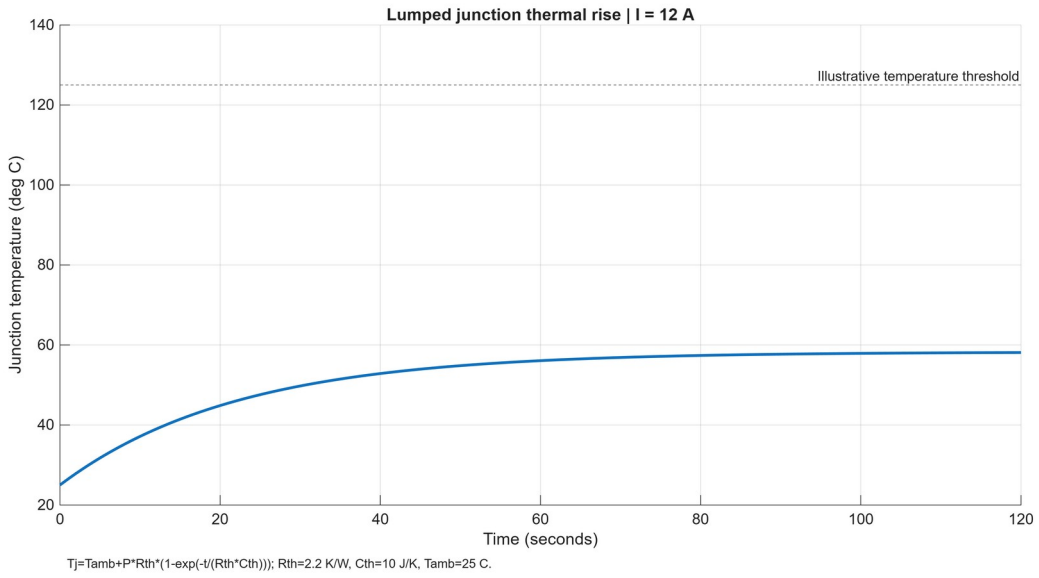
REFERENCE illustration SCR_050. Reproduce as SCR_050.png with the supplied MATLAB script.

Figure SCR_051 — Lumped junction thermal response



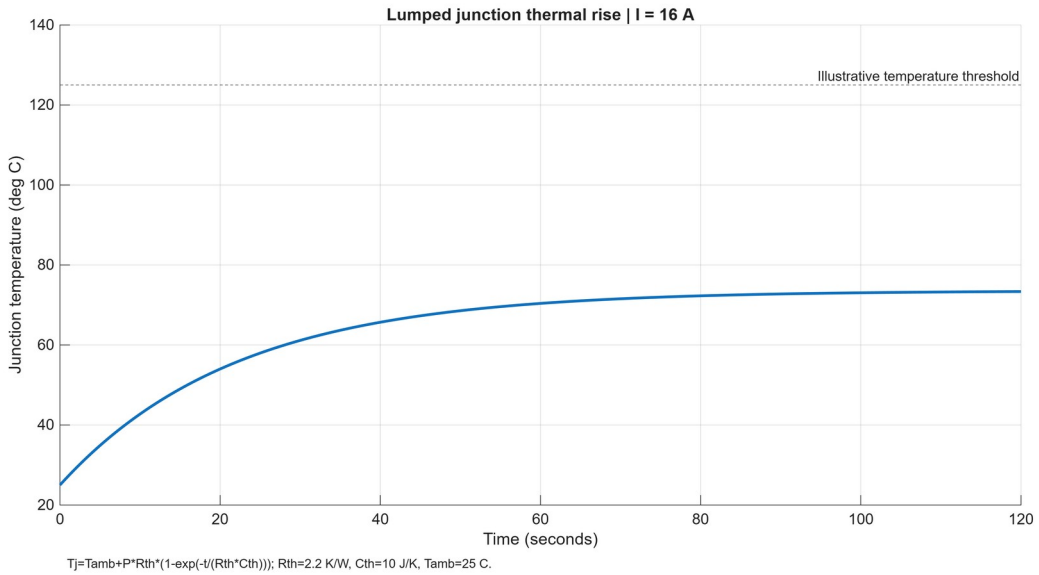
REFERENCE illustration SCR_051. Reproduce as SCR_051.png with the supplied MATLAB script.

Figure SCR_052 — Lumped junction thermal response



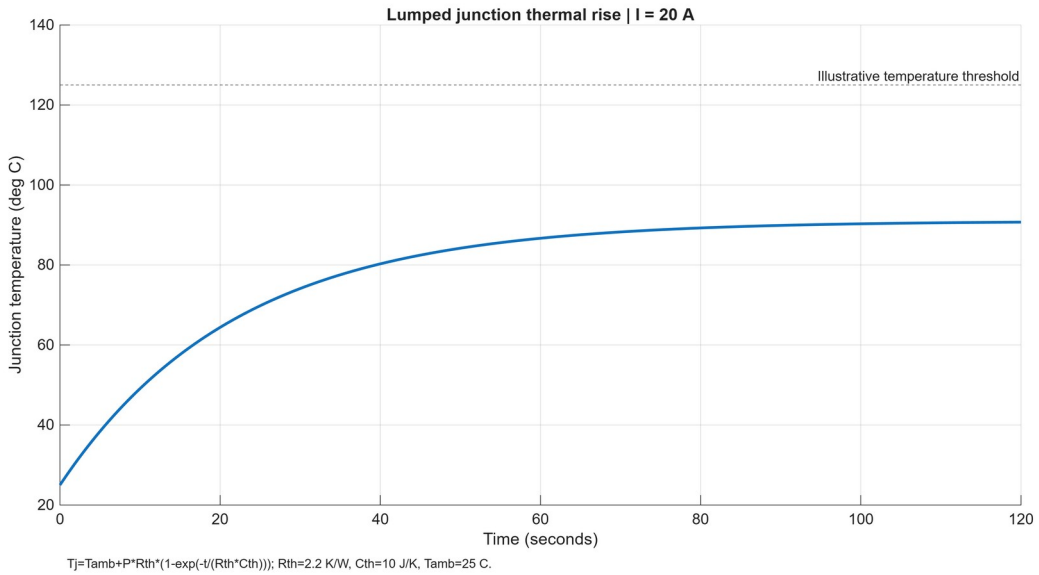
REFERENCE illustration SCR_052. Reproduce as SCR_052.png with the supplied MATLAB script.

Figure SCR_053 — Lumped junction thermal response



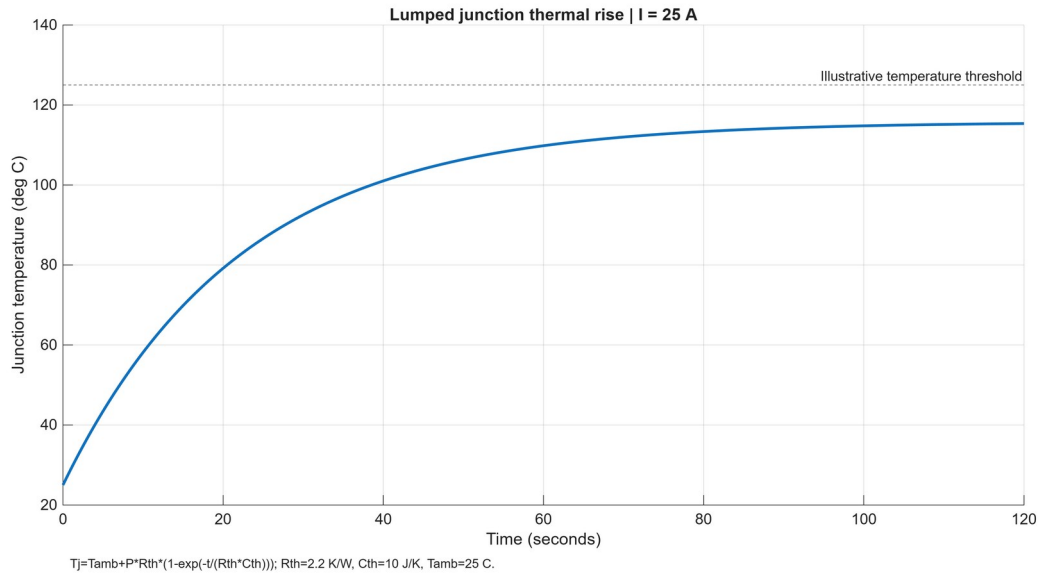
REFERENCE illustration SCR_053. Reproduce as SCR_053.png with the supplied MATLAB script.

Figure SCR_054 — Lumped junction thermal response



REFERENCE illustration SCR_054. Reproduce as SCR_054.png with the supplied MATLAB script.

Figure SCR_055 — Lumped junction thermal response

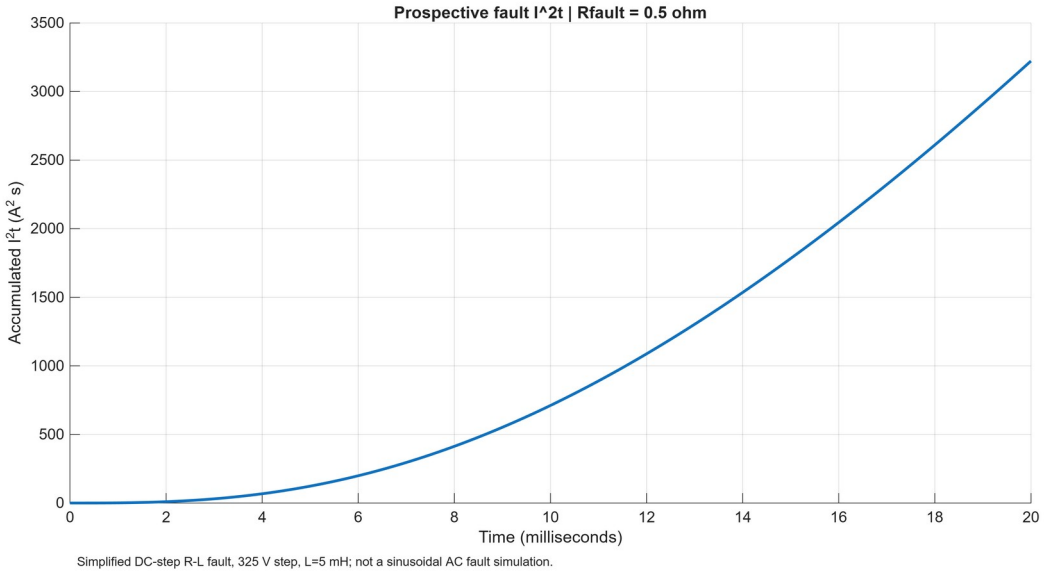


REFERENCE illustration SCR_055. Reproduce as SCR_055.png with the supplied MATLAB script.

056–061 | Prospective fault I²t

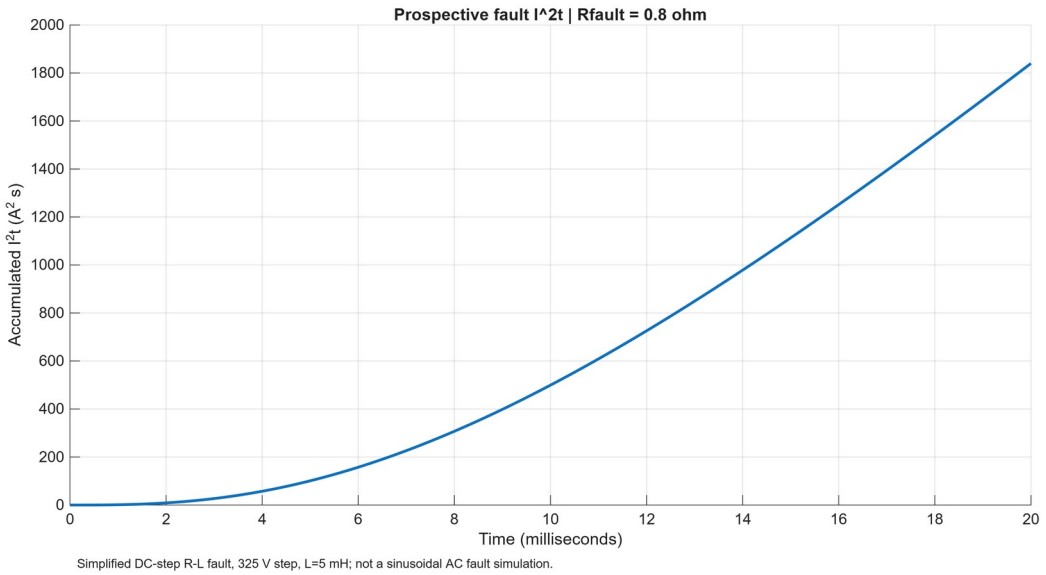
DC-step R–L fault current integrated over time; not a sinusoidal AC fault model.

Figure SCR_056 — Prospective fault I²t



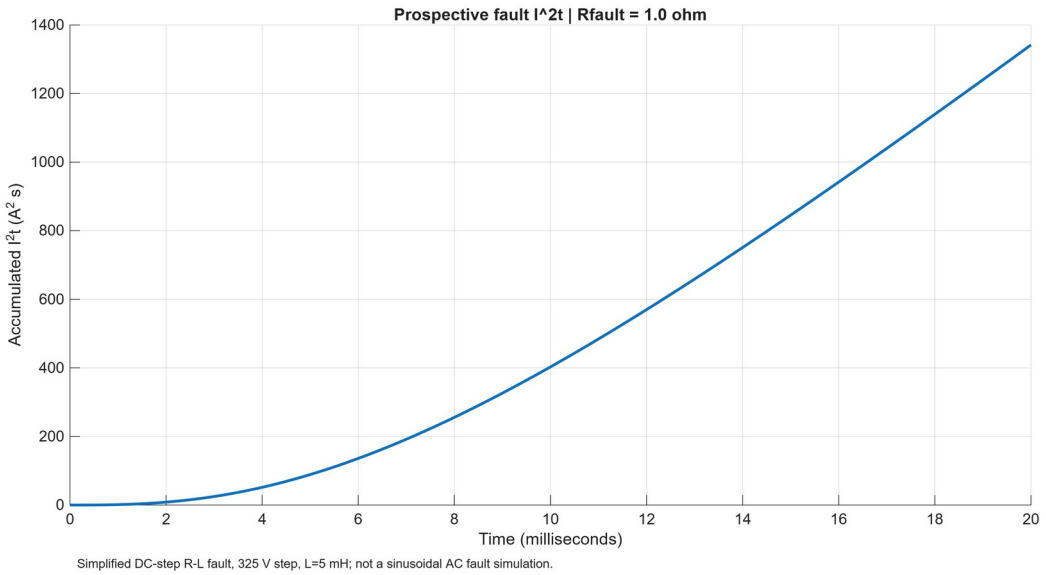
REFERENCE illustration SCR_056. Reproduce as SCR_056.png with the supplied MATLAB script.

Figure SCR_057 — Prospective fault I^2t



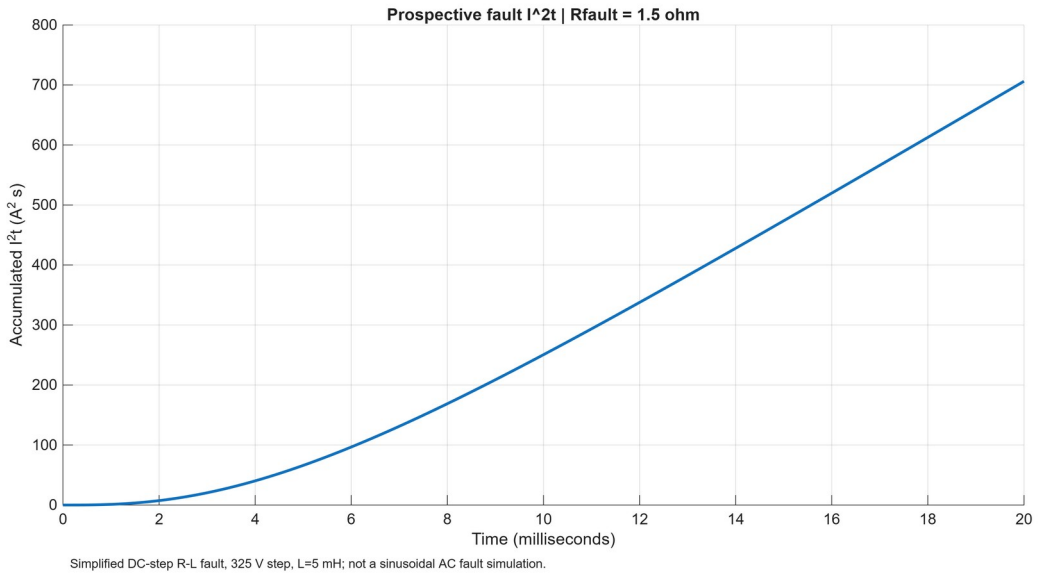
REFERENCE illustration SCR_057. Reproduce as SCR_057.png with the supplied MATLAB script.

Figure SCR_058 — Prospective fault I^2t



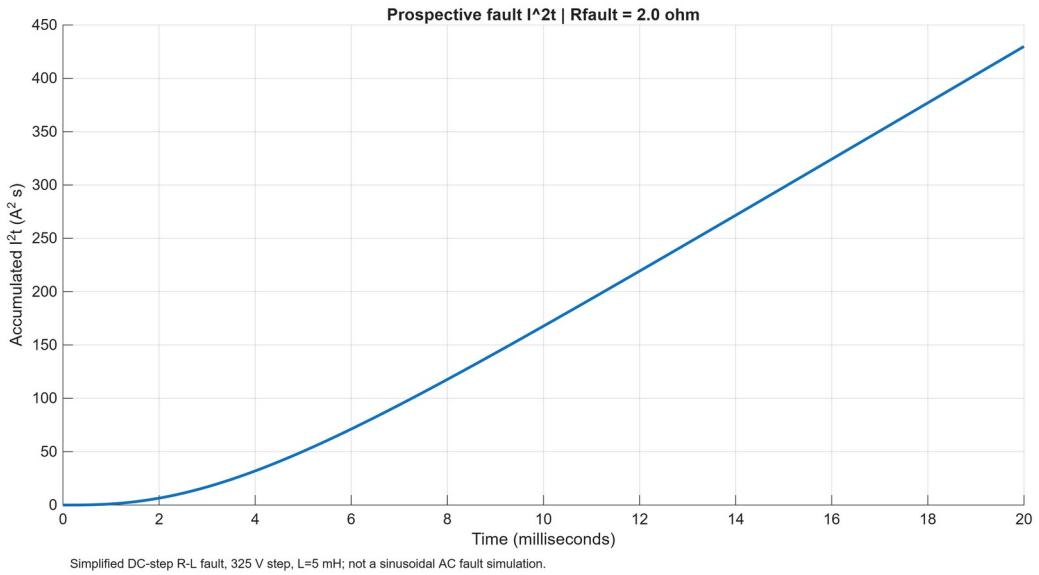
REFERENCE illustration SCR_058. Reproduce as SCR_058.png with the supplied MATLAB script.

Figure SCR_059 — Prospective fault I^2t



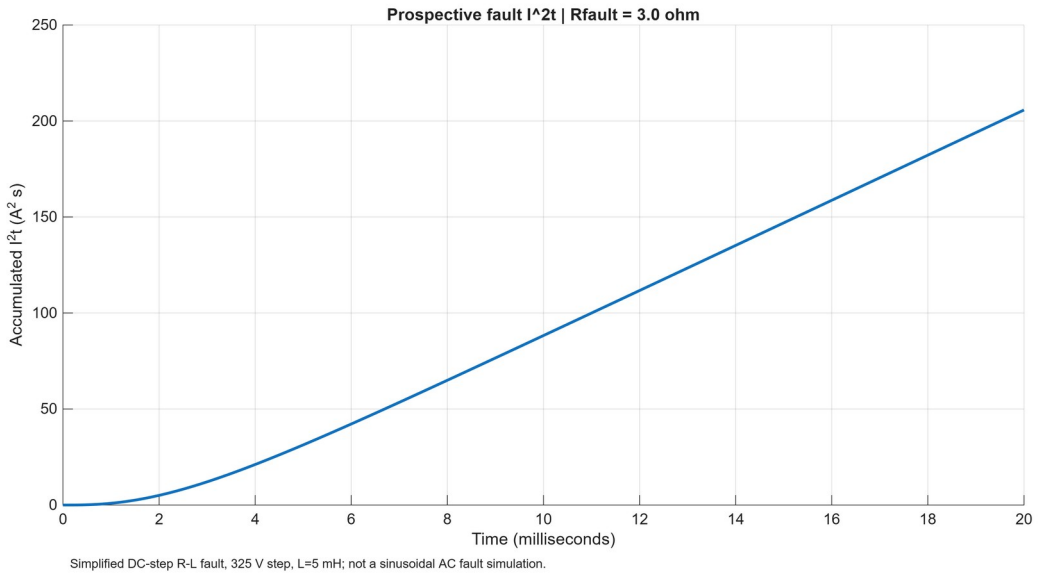
REFERENCE illustration SCR_059. Reproduce as SCR_059.png with the supplied MATLAB script.

Figure SCR_060 — Prospective fault I^2t



REFERENCE illustration SCR_060. Reproduce as SCR_060.png with the supplied MATLAB script.

Figure SCR_061 — Prospective fault I^2t

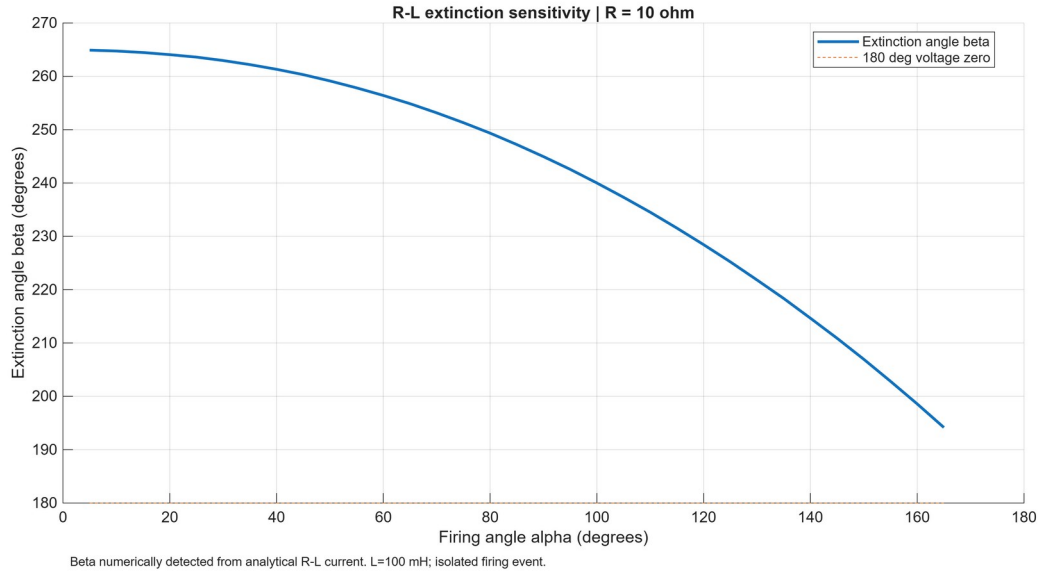


REFERENCE illustration SCR_061. Reproduce as SCR_061.png with the supplied MATLAB script.

062–065 | Extinction-angle sensitivity

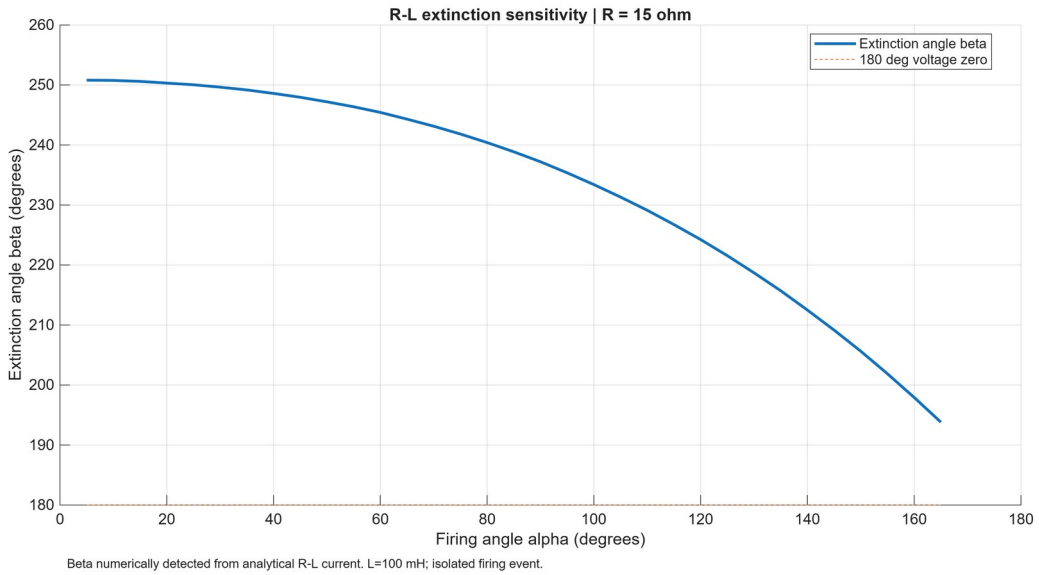
Numerically detected extinction angle beta against firing angle alpha at varied resistance.

Figure SCR_062 — Extinction-angle sensitivity



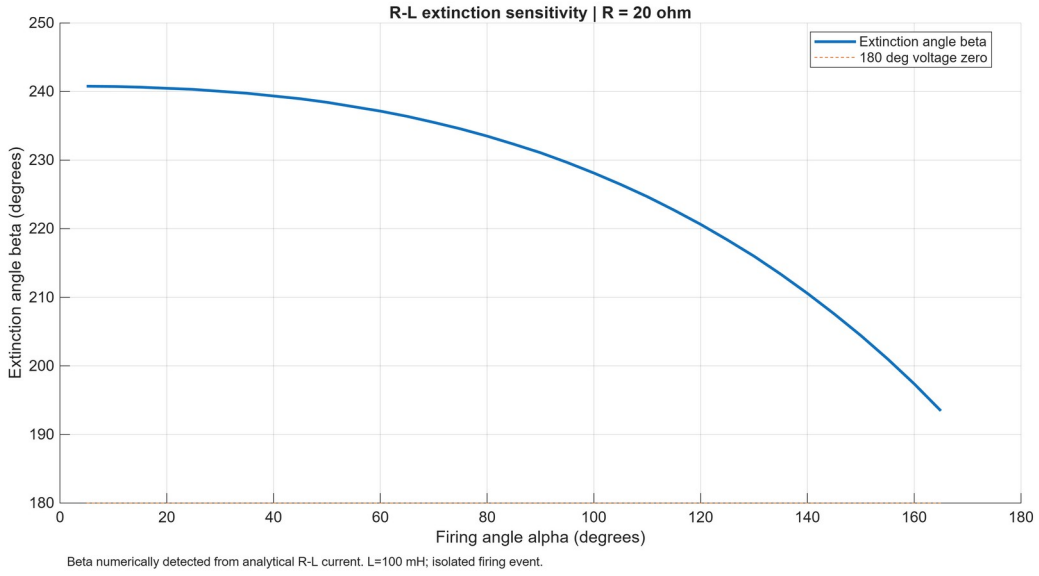
REFERENCE illustration SCR_062. Reproduce as SCR_062.png with the supplied MATLAB script.

Figure SCR_063 — Extinction-angle sensitivity



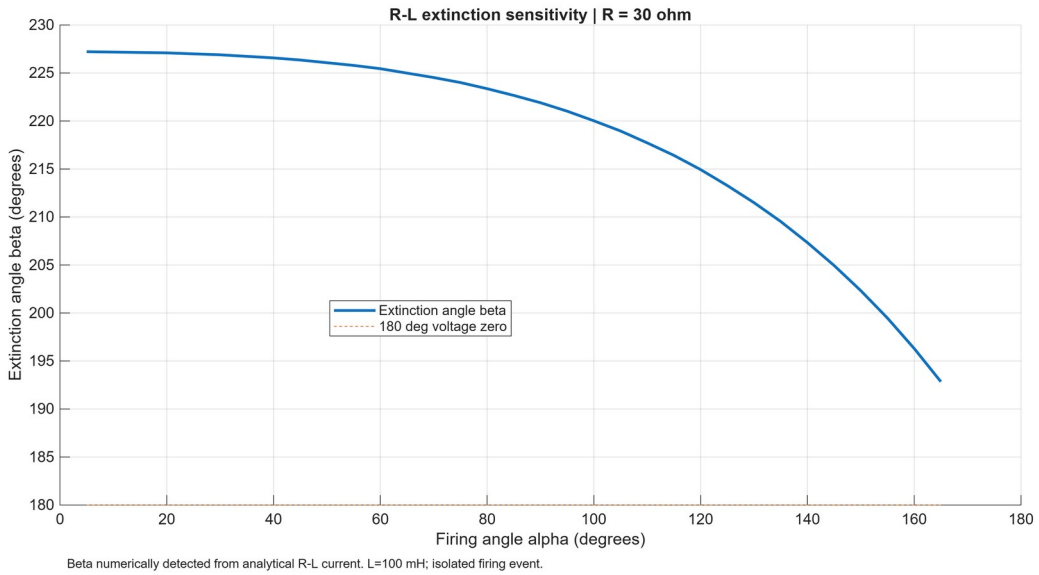
REFERENCE illustration SCR_063. Reproduce as SCR_063.png with the supplied MATLAB script.

Figure SCR_064 — Extinction-angle sensitivity



REFERENCE illustration SCR_064. Reproduce as SCR_064.png with the supplied MATLAB script.

Figure SCR_065 — Extinction-angle sensitivity



REFERENCE illustration SCR_065. Reproduce as SCR_065.png with the supplied MATLAB script.

4. Reproducibility, limitations and verification

• Before publication as executed MATLAB results: run all 65 figures in MATLAB Online; confirm output file count and complete RUN_REPORT; visually compare the generated results with these references.

• Validation record: The project author reports a successful complete run in MATLAB Online, generating all 65 engineering graphs. The earlier independent closure review separately recreated figures 001–013 with Python; it did not repeat the complete 65-graph run in native MATLAB. These statements concern different verification activities.

• Model boundaries: idealized switching/conduction, approximate latching, analytical R–L current, simplified R–L–C topology, lumped thermal response, and DC-step fault model. Do not use for SCR part selection without checking datasheets, tolerances, thermal impedance and protection coordination.

• Check important parameters: I_GT, I_L, I_H, V_T, dv/dt, di/dt, t_q, junction thermal impedance and surge I^2t against the specific SCR datasheet; these are not universally modeled here.

Appendix A — Complete MATLAB source (commented)

The following is the exact text of SCR11_v6_REVISED_commented.m included in this publication ZIP. For execution use the separate .m file, not text copied from a PDF.

```
%% AHDS SCR MATLAB Simulation & Analysis — 65 Engineering Figures
% AHDS — Anissoff Hardware Design System
% Design once. Reuse forever.
% Contact: ahds@engineer.com
% Version 1.3 (commented, v6 package) | Base MATLAB | MATLAB Online (R2021b or newer recommended)
%
% WHAT CHANGED IN 1.3 (compared with 1.2):
%   Version 1.2 wrote the 65 PNGs directly into the synced MATLAB Drive folder.
%   Its ZIP then held 0-byte files for 64 of the 65 images.
%   This version creates all PNGs in a local scratch folder (tempdir), zips there,
%   unzips the ZIP to check that every PNG is a valid image, and only then
%   copies the single ZIP into your Drive folder.
%   Version 1.3 (commented): code is unchanged; only comments were corrected/added.
%
% CHANGES IN THE v6 PACKAGE (compared with the earlier 1.3 script):
%   - Figures 1-6: voltage axis extended to 2.35 V so the 40 A reference line (Vt = 2.25 V) is fully visible.
```

```
%   - Figures 7-13: coupled latch model. The gate is a 10-35 us pulse; the anode current latches only if the
%     normalized current reaches IL/Iinfinity = 0.80 while the gate is still on (otherwise it collapses at 35 us).
%   - Comment-only edit after v6: every line of the gate/latch group is now commented in English and the stale
%     "2.2 V" and "two curves" comments are corrected. No executable statement was changed.
%
% PURPOSE:
%   Generate 65 numbered, annotated SCR engineering figures in one run.
%   Each figure is drawn (hidden by default), saved as a 300 DPI PNG, then closed.
%   All PNGs plus RUN_REPORT.txt are packed into one ZIP.
%   Results are MODEL predictions, NOT validated device datasheet curves.
%
% MODELS USED (what each group of figures really plots):
%   Figures 1-6   : on-state V-I curve (threshold + slope), clipped at a current limit.
%   Figures 7-13  : simplified gate-enabled latch model and anode-current rise (illustrative).
%   Figures 14-21 : ideal phase-controlled half-wave rectifier, resistive load.
%   Figures 22-31 : analytical R-L load current and natural extinction.
%   Figures 32-41 : series R-L-C capacitor discharge (snubber proxy), sweep of C.
%   Figures 42-48 : turn-off time margin = available reverse time - assumed tq.
%   Figures 49-55 : first-order lumped thermal model (Rth, Cth) for junction temperature.
%   Figures 56-61 : prospective fault current (DC-step R-L) and its I^2*t.
%   Figures 62-65 : extinction angle beta versus firing angle alpha for four resistances.
%
% QUICK START:
%   Upload this ONE .m file to MATLAB Online, open it, and click Run.
%   The finished ZIP is copied to your Drive folder (the path is printed at the end).
%   Set SHOW_SECONDS = 0 to remove the extra delay between figures.
%   To test quickly, set FIGURE_COUNT = 3 and check SCR_001..003.png.
%
% IMPORTANT: All helper functions are embedded at the bottom of this file.


clearvars;                          % Delete all variables from the workspace before starting.
clc;                                % Clear the command window text.
SHOW_SECONDS = 0.6;                 % Extra delay (s) after drawing each figure; only visible on screen if FIG_VISIBLE = 'on'.
EXPORT_PAUSE = 0.5;                 % Seconds to wait before/after each export step so the file can finish being written.
FIG_VISIBLE = 'off';                % 'off' = draw figures in memory (more reliable in MATLAB Online); 'on' = show them on screen.
OUTPUT_DIR = 'AHDS_SCR_Results';    % Name of the folder that will hold the PNGs and the run report.
ARCHIVE_NAME = 'AHDS_SCR_65_Graphs.zip';  % Name of the single ZIP file that is delivered at the end.
FIGURE_COUNT = 65;                  % Number of figures to generate (65 = full set; use 3 for a quick test).
finalFolder = ahdsWritableFolder();     % Find the folder where the finished ZIP will be delivered (usually your MATLAB Drive).
workRoot = fullfile(tempdir, 'AHDS_SCR_work'); % Path of a local scratch folder (not synced) used while generating files.
if ~exist(workRoot, 'dir')          % Check whether the scratch folder is missing.
    mkdir(workRoot);                % Create the scratch folder.
end                                 % End of the scratch-folder check.
cd(workRoot);                       % Move into the scratch folder so all PNGs are created there.
fprintf('Generating in local scratch folder: %s\n', pwd); % Print the scratch folder path.
fprintf('Final ZIP will be delivered to:    %s\n', finalFolder); % Print where the final ZIP will be copied.
if ~exist(OUTPUT_DIR, 'dir')        % Check whether the results folder already exists in the scratch folder.
    [mkOK, mkMsg] = mkdir(OUTPUT_DIR);      % Create the results folder and keep the success flag and message.
    if ~mkOK                        % Check whether the folder could not be created.
        error('AHDS:Output', 'Cannot create folder %s in %s: %s', OUTPUT_DIR, pwd, mkMsg); % Stop with a clear message.
    end                             % End of the creation-failure check.
end                                 % End of the results-folder check.
oldPNGs = dir(fullfile(OUTPUT_DIR, 'SCR_*.png')); % List PNG files left over from an earlier run.
for k = 1:numel(oldPNGs)            % Loop over each leftover PNG.
    delete(fullfile(OUTPUT_DIR, oldPNGs(k).name)); % Delete it so old images cannot be mistaken for new ones.
end                                 % End of the cleanup loop.
try                                 % Start a protected block: a failed delete must not stop the run.
    if exist(ARCHIVE_NAME, 'file') == 2     % Check whether an old ZIP file exists here.
        delete(ARCHIVE_NAME);       % Delete the old ZIP.
    end                             % End of the old-ZIP check.
catch                               % If the delete fails (for example the file is locked)...
end                                 % ...ignore the problem and continue.
logPath = fullfile(OUTPUT_DIR, 'RUN_REPORT.txt'); % Build the path of the run report text file.
fid = fopen(logPath, 'w');          % Open the run report for writing (fid < 0 means it failed).
if fid < 0                          % Check whether the report file could not be opened.
    error('AHDS:Output', 'Cannot create the run report.'); % Stop, because the run cannot be logged.
end                                 % End of the report-file check.
fprintf(fid, 'AHDS SCR 65 Graphs — Simulation Report\n'); % Write the report title line.
fprintf(fid, 'Illustrative engineering models; not device certification data.\n\n'); % Write the scope statement and a blank line.
completed = false(1, FIGURE_COUNT);     % Create a flag per figure, all false until the figure is exported.
failedMessages = cell(1, FIGURE_COUNT);     % Create a slot per figure for an error message (stays in the workspace for inspection).
for figureID = 1:FIGURE_COUNT       % Loop over the figures in order.
    fig = [];                       % Clear the figure handle so the cleanup below is safe if figure() fails.
    try                             % Start a protected block: one failing figure must not stop the batch.
        fig = figure('Color', 'w', 'Name', sprintf('AHDS SCR %03d', figureID), ...
            'NumberTitle', 'off', 'Visible', FIG_VISIBLE, 'Position', [100 100 1100 650]); % Create a white 1100x650 figure, hidden or shown per FIG_VISIBLE.
        ax = axes(fig);             % Create one explicit axes in this figure to draw on.
        hold(ax, 'on');             % Let several lines be drawn on the same axes.
        [plotTitle, notes] = ahdsPlotSCR(figureID, ax); % Compute and draw this figure's curves; get its title and note text.
        title(ax, plotTitle, 'Interpreter', 'none', 'FontWeight', 'bold'); % Put the bold title on the axes (plain text, no TeX).
        xlabel(ax, ahdsXLabel(figureID));   % Set the horizontal axis label (with units) for this figure.
        ylabel(ax, ahdsYLabel(figureID));   % Set the vertical axis label (with units) for this figure.
        grid(ax, 'on');             % Show grid lines to make values easier to read.
        set(ax, 'FontSize', 11);        % Use font size 11 for axis numbers and labels.
        ax.Position = [0.09 0.16 0.84 0.71]; % Set the axes box [left bottom width height], leaving space below for the note.
        annotation(fig, 'textbox', [0.09 0.035 0.84 0.055], 'String', notes, ...
```

```
        'EdgeColor', 'none', 'Interpreter', 'none', 'FontSize', 9); % Add the model note under the plot (no border, plain text, size 9).
    drawnow;                    % Force MATLAB to finish drawing before continuing.
    if SHOW_SECONDS > 0         % Check whether an extra delay is requested.
      pause(SHOW_SECONDS);      % Wait SHOW_SECONDS seconds.
    end                         % End of the delay check.
    filename = sprintf('SCR_%03d.png', figureID); % Build the file name, for example SCR_007.png.
    filepath = fullfile(OUTPUT_DIR, filename); % Build the full path of the PNG inside the results folder.
    drawnow;                    % Force pending drawing to finish again just before exporting.
    method = ahdsExportFigure(fig, filepath, EXPORT_PAUSE); % Export the PNG, trying several methods and checking that curves are present; get the method used.
    fileInfo = dir(filepath);   % Read the file information (including size) of the exported PNG.
    if isempty(fileInfo) || fileInfo.bytes == 0 % Check whether the PNG is missing or empty.
      if exist(filepath, 'file')  % Check whether an empty file exists.
        delete(filepath);       % Delete the empty file before trying again.
      end                       % End of the empty-file check.
      drawnow;                  % Make sure drawing is finished before the last-resort export.
      pause(EXPORT_PAUSE);      % Wait before the last-resort export.
      print(fig, filepath, '-dpng', '-r300'); % Last-resort export with print at 300 DPI.
      pause(EXPORT_PAUSE);      % Wait for that export to finish.
      fileInfo = dir(filepath);   % Read the file information again after the last-resort export.
    end                         % End of the missing/empty check.
    if isempty(fileInfo) || fileInfo.bytes == 0 % Check whether the PNG is still missing or empty.
      error('AHDS:Export', 'PNG is missing or empty after both export methods.'); % Raise an error so this figure is logged as failed.
    end                         % End of the second missing/empty check.
    completed(figureID) = true;     % Mark this figure as successfully exported.
    fprintf(fid, 'OK   %03d | %s | %s\n', figureID, filename, plotTitle); % Write an OK line (number, file, title) in the report.
    fprintf(fid, '     method: %s\n', method); % Write which export method produced this image.
    fprintf('Saved %02d/%02d: %s (%s)\n', figureID, FIGURE_COUNT, filename, method); % Print progress in the command window.
  catch ME                      % If anything above failed, catch the error in ME.
    failedMessages{figureID} = ME.message; % Store the error text for this figure.
    fprintf(fid, 'FAIL %03d | %s\n', figureID, ME.message); % Write a FAIL line with the reason in the report.
    warning('AHDS:PlotFailed', 'Graph %03d: %s', figureID, ME.message); % Show a warning in the command window.
  end                           % End of the protected block.
  if ~isempty(fig) && isgraphics(fig)   % Check whether the figure still exists.
    close(fig);                 % Close it to free memory before the next figure.
  end                           % End of the close check.
end                             % End of the loop over figures.
successCount = sum(completed);          % Count how many figures were exported successfully.
fprintf(fid, '\nGenerated: %d / %d PNGs\n', successCount, FIGURE_COUNT); % Write the total count in the report.
if successCount == FIGURE_COUNT         % Check whether every figure succeeded.
  fprintf(fid, 'STATUS: COMPLETE\n');   % Write STATUS: COMPLETE in the report.
else                            % Otherwise some figures failed.
  fprintf(fid, 'STATUS: INCOMPLETE — inspect failures above.\n'); % Write STATUS: INCOMPLETE in the report.
end                             % End of the status check.
fclose(fid);                    % Close the report file so everything is written to disk.
if successCount == FIGURE_COUNT         % Only build the ZIP if all figures were exported.
  zip(ARCHIVE_NAME, OUTPUT_DIR);        % Create the ZIP of the results folder in the scratch folder.
  chkDir = fullfile(workRoot, 'zip_verify'); % Path of a temporary folder used to test the ZIP.
  if exist(chkDir, 'dir')           % Check whether an older test folder exists.
    rmdir(chkDir, 's');             % Delete it together with its contents.
  end                               % End of the old-test-folder check.
  mkdir(chkDir);                    % Create an empty test folder.
  unzip(ARCHIVE_NAME, chkDir);      % Extract the ZIP into the test folder, as a user would.
  zPngs = dir(fullfile(chkDir, '**', 'SCR_*.png')); % List every PNG found in the extracted files (all subfolders).
  zEmpty = sum([zPngs.bytes] == 0);     % Count extracted PNGs whose size is 0 bytes.
  zBad = 0;                         % Start the count of PNGs that cannot be read as images.
  for k = 1:numel(zPngs)            % Loop over each extracted PNG.
    try                             % Start a protected block for reading the image.
      imread(fullfile(zPngs(k).folder, zPngs(k).name)); % Try to decode the PNG; this fails if the file is damaged.
    catch                           % If decoding failed...
      zBad = zBad + 1;              % ...count this file as unreadable.
    end                             % End of the read attempt.
  end                               % End of the loop over extracted PNGs.
  if numel(zPngs) ~= FIGURE_COUNT || zEmpty > 0 || zBad > 0 % Check whether the ZIP is incomplete, has empty files, or has damaged files.
    error('AHDS:ZipCheck', 'ZIP check FAILED: %d PNGs found, %d empty, %d unreadable.', numel(zPngs), zEmpty, zBad); % Stop and report the numbers.
  end                               % End of the ZIP check.
  rmdir(chkDir, 's');               % Delete the temporary test folder.
  destZip = fullfile(finalFolder, ARCHIVE_NAME); % Path where the ZIP will be delivered.
  srcBytes = dir(ARCHIVE_NAME).bytes;   % Size in bytes of the verified local ZIP.
  copyfile(ARCHIVE_NAME, destZip, 'f');   % Copy the ZIP to the delivery folder (overwrite if it exists).
  delivered = false;                % Start by assuming the copy is not yet confirmed.
  for tries = 1:20                  % Check up to 20 times (about 20 seconds).
    pause(1);                       % Wait one second so a synced folder can finish writing.
    d = dir(destZip);               % Read the information of the delivered file.
    if ~isempty(d) && d.bytes == srcBytes % Check that the file exists and has the same size as the verified ZIP.
      delivered = true;             % Record that delivery is confirmed.
      break;                        % Stop waiting.
    end                             % End of the size comparison.
  end                               % End of the waiting loop.
  if ~delivered                     % Check whether the copy was never confirmed.
    warning('AHDS:Deliver', 'ZIP copy to %s may be incomplete. Wait a minute and check its size (%d bytes).', finalFolder, srcBytes); % Warn the user with the expected size.
  end                               % End of the delivery check.
  fprintf('\nAHDS: %d/%d PNGs exported and VERIFIED INSIDE THE ZIP (%d bytes).\n', successCount, FIGURE_COUNT, srcBytes); % Print the verified summary.
  fprintf('ZIP delivered to: %s\n', destZip); % Print where to download the ZIP.
else                                % Otherwise at least one figure failed.
  fprintf('\nAHDS: %d/%d PNGs exported; ZIP NOT created. See RUN_REPORT.txt.\n', successCount, FIGURE_COUNT); % Say that no ZIP was made.
```

```
end                                 % End of the ZIP section.
if successCount ~= FIGURE_COUNT           % Check again whether some figures failed.
   warning('AHDS:Incomplete', 'ZIP contains an incomplete set; see RUN_REPORT.txt.'); % Warn that the set is incomplete (no ZIP was created in this case).
end                                 % End of the final check.


%% Plot engine: one function implements all 65 engineering plots.
function [plotTitle, notes] = ahdsPlotSCR(id, ax)
% AHDSPlotSCR: select and draw the plot for figure number id on the axes ax.
% INPUT: id — integer figure index from 1 to 65.
%        ax — axes handle to draw on (hold is already on).
% OUTPUTS: plotTitle — figure title text; notes — short text describing the model.
% This function uses Base MATLAB only; no Simulink or toolbox is required.


f = 50;                             % Mains frequency in Hz.
w = 2*pi*f;                         % Angular frequency in rad/s (about 314 rad/s).
Vrms = 230;                         % Supply voltage, RMS value in volts.
Vm = sqrt(2)*Vrms;                  % Supply peak voltage in volts (about 325 V).
R = 20;                             % Load resistance in ohms (used by the R-L figures 22-31).
L = 0.10;                           % Load inductance in henries, i.e. 100 mH (used by figures 22-31 and 62-65).
notes = 'Illustrative model. Verify device-specific limits against the datasheet.'; % Default note, replaced in each group below.
hold(ax, 'on');                     % Make sure several lines can be drawn on the same axes.


if id <= 6                          % GROUP 1 (figures 1-6): on-state V-I curve.
   v = linspace(0, 2.35, 400);      % 400 voltage points from 0 to 2.35 V.
   currents = [1 2 5 10 20 40];     % Current limit (A) used for each of the 6 figures.
   Imax = currents(id);             % Current limit for this figure.
   Vt = 0.85 + 0.035*Imax;          % Voltage at which the illustrative law reaches the chosen current limit.
   i = max(0, (v - 0.85)/0.035);    % Current: zero below 0.85 V, then rises linearly with slope 1/0.035 A per volt.
   plot(ax, v, min(i,Imax), 'LineWidth', 2); % Draw the curve, clipped at the current limit Imax.
   plot(ax, [Vt Vt], [0 Imax], '--', 'LineWidth', 1); % Draw a dashed vertical reference line at voltage Vt.
   plotTitle = sprintf('SCR on-state V-I model | current range %g A', Imax); % Title showing the current limit.
   notes = 'Piecewise illustrative on-state law: I=max(0,(V-0.85)/0.035); not a datasheet fit.'; % Note giving the formula used.
elseif id <= 13                     % GROUP 2 (figures 7-13): gate pulse and turn-on.
   t = linspace(0, 120e-6, 1200);   % 1200 time points from 0 to 120 microseconds.
   tr = [1 2 3 5 8 12 18]*1e-6;     % Seven turn-on time constants, converted from microseconds to seconds.
   tau = tr(id-6);                  % Time constant for this figure.
   % Simplified gate-enabled latching model; dimensionless illustrative thresholds.
   % Gate is a rectangular pulse from 10 us to 35 us; IL/Iinfinity = 0.80.
   gate = double(t >= 10e-6 & t <= 35e-6); % Gate pulse: 1 while 10 us <= t <= 35 us, otherwise 0.
   ILnorm = 0.80;                   % Normalized latching current IL/Iinfinity (dimensionless, illustrative).
   candidate = (1-exp(-max(t-10e-6,0)/tau)).*(t>=10e-6); % First-order current rise starting at 10 us with time constant tau (normalized to its final value).
   latchIndex = find(candidate >= ILnorm & gate > 0, 1, 'first'); % First sample where the current reaches IL while the gate is still on (empty = never latches).
   turnon = zeros(size(t));         % Pre-allocate the anode-current curve with zeros.
   if ~isempty(latchIndex)          % Latching happened before the gate pulse ended.
      turnon = candidate; % Latching condition satisfied before gate removal.
   else                             % Latching did not happen in time.
      turnon = candidate.*gate; % Fails to latch; simplified forced extinction.
   end
   plot(ax, t*1e6, gate, 'LineWidth', 1.8);   % Draw the gate pulse against time in microseconds.
   plot(ax, t*1e6, turnon, 'LineWidth', 1.8); % Draw the anode-current rise against time in microseconds.
   plot(ax, t*1e6, ILnorm*ones(size(t)), '--', 'LineWidth', 1); % Dashed horizontal reference line at the normalized latching level IL/Iinfinity.
   legend(ax, 'Gate pulse (normalized)', 'Anode current (normalized)', 'Illustrative IL / Iinfinity', 'Location', 'best'); % Add a legend naming the three curves (gate, anode
current, IL level).
   plotTitle = sprintf('Gate excitation and turn-on | tau = %.0f us', tau*1e6); % Title showing tau in microseconds.
   notes = 'Illustrative coupled latch: gate 10-35 us; IL/Iinf=0.80; turn-on tau varied. Not a device-physics model.'; % Note about the model limits.
elseif id <= 21                     % GROUP 3 (figures 14-21): half-wave control, resistive load.
   alphaDeg = [0 15 30 45 60 90 120 150];  % Firing angles in degrees for the 8 figures.
   alpha = alphaDeg(id-13)*pi/180;  % Firing angle for this figure, converted to radians.
   theta = linspace(0, 2*pi, 2400); % 2400 angle points covering one full cycle.
   vs = Vm*sin(theta);              % Source voltage over the cycle.
   conduct = (theta>=alpha)&(theta<=pi);   % True where the ideal SCR conducts (from alpha to pi).
   vo = vs.*conduct;                % Load voltage: source voltage where conducting, zero elsewhere.
   plot(ax, theta*180/pi, vs, '--', 'LineWidth', 1.2); % Draw the source voltage as a dashed line, angle in degrees.
   plot(ax, theta*180/pi, vo, 'LineWidth', 2); % Draw the load voltage as a solid line.
   legend(ax, 'Source voltage', 'Load voltage', 'Location', 'best'); % Add a legend naming the two curves.
   plotTitle = sprintf('R-load half-wave control | firing angle = %d deg', alphaDeg(id-13)); % Title showing the firing angle in degrees.
   notes = 'Ideal SCR, pure R load, no freewheel path; voltage is zero outside conduction.'; % Note about the assumptions.
elseif id <= 31                     % GROUP 4 (figures 22-31): R-L load, current and extinction.
   alphaDeg = [15 30 45 60 75 90 105 120 135 150]; % Firing angles in degrees for the 10 figures.
   alpha = alphaDeg(id-21)*pi/180;  % Firing angle for this figure, converted to radians.
   theta = linspace(0, 4*pi, 5000); % 5000 angle points covering two cycles (0 to 720 degrees).
   current = ahdsRLCurrent(theta, alpha, Vm, R, L, w); % SCR current from the analytical R-L solution.
   plot(ax, theta*180/pi, current, 'LineWidth', 2); % Draw the SCR current (zero when not conducting).
   plot(ax, theta*180/pi, Vm*sin(theta)/R, '--', 'LineWidth', 1); % Draw, dashed, the current a pure resistor R would carry (reference only).
   legend(ax, 'R-L SCR current', 'R-only reference', 'Location', 'best'); % Add a legend naming the two curves.
   plotTitle = sprintf('R-L current and natural extinction | alpha = %d deg', alphaDeg(id-21)); % Title showing the firing angle in degrees.
   notes = sprintf('R=%g ohm, L=%g H, 230 Vrms, 50 Hz. Extinction is the first post-firing current zero.',R,L); % Note with the parameter values.
elseif id <= 41                     % GROUP 5 (figures 32-41): series R-L-C discharge (snubber proxy).
   Cvals = [10 22 33 47 68 100 150 220 330 470]*1e-9; % Ten capacitor values, converted from nanofarads to farads.
   C = Cvals(id-31);                % Capacitance for this figure.
   Rs = 100;                        % Series resistance in ohms.
   Ls = 50e-6;                      % Series inductance in henries (50 microhenries).
   t = linspace(0, 150e-6, 2500);   % 2500 time points from 0 to 150 microseconds.
   [vc, ic] = ahdsRLCTransient(t, 400, Rs, Ls, C); % Capacitor voltage and loop current, starting from 400 V and zero current.
   plot(ax, t*1e6, vc, 'LineWidth', 2);    % Draw the capacitor voltage against time in microseconds.
```

```
    plot(ax, t*1e6, ic*Rs, '--', 'LineWidth', 1.2); % Draw, dashed, the resistor voltage (current times Rs).
    legend(ax, 'Capacitor voltage', 'Series resistor voltage', 'Location', 'best'); % Add a legend naming the two curves.
    plotTitle = sprintf('Snubber RLC transient | C = %.0f nF', C*1e9); % Title showing the capacitance in nanofarads.
    notes = 'Series RLC initial capacitor discharge: Vc(0)=400 V, i(0)=0, Rs=100 ohm, Ls=50 uH. Simplified topology.'; % Note with the parameter values.
elseif id <= 48                         % GROUP 6 (figures 42-48): turn-off time margin.
    tqUs = [20 35 50 70 90 120 160];    % Assumed required turn-off times tq in microseconds for the 7 figures.
    tq = tqUs(id-41);                   % Required turn-off time for this figure.
    available = linspace(0, 250, 600);  % 600 values of available reverse-bias time, 0 to 250 microseconds.
    margin = available - tq;            % Margin: positive when the available time exceeds tq.
    plot(ax, available, margin, 'LineWidth', 2); % Draw the margin against the available time.
    yline(ax, 0, '--', 'Zero margin');  % Draw a dashed horizontal line at zero margin, the pass/fail limit.
    plotTitle = sprintf('Turn-off recovery margin | tq = %d us', tq); % Title showing tq in microseconds.
    notes = 'Margin=t_reverse_available-tq; tq is an assumed datasheet-dependent requirement, not trr.'; % Note explaining the formula.
elseif id <= 55                         % GROUP 7 (figures 49-55): junction temperature rise.
    Ivals = [2 5 8 12 16 20 25];        % Average conduction currents in amperes for the 7 figures.
    I = Ivals(id-48);                   % Current for this figure.
    t = linspace(0, 120, 1000);         % 1000 time points from 0 to 120 seconds.
    Rth = 2.2;                          % Thermal resistance in K/W.
    Cth = 10;                           % Thermal capacitance in J/K.
    P = 0.9*I + 0.03*I^2;               % Conduction power loss in watts (simple assumed formula).
    Tj = 25 + P*Rth*(1-exp(-t/(Rth*Cth))); % Junction temperature: 25 C ambient plus a first-order rise with time constant Rth*Cth.
    plot(ax, t, Tj, 'LineWidth', 2);    % Draw the junction temperature against time in seconds.
    yline(ax, 125, '--', 'Illustrative temperature threshold'); % Draw a dashed line at 125 C as a reference level only.
    plotTitle = sprintf('Lumped junction thermal rise | I = %g A', I); % Title showing the current.
    notes = 'Tj=Tamb+P*Rth*(1-exp(-t/(Rth*Cth))); Rth=2.2 K/W, Cth=10 J/K, Tamb=25 C.'; % Note giving the formula and values.
elseif id <= 61                         % GROUP 8 (figures 56-61): fault current and I^2*t.
    Rfaults = [0.5 0.8 1 1.5 2 3];      % Assumed fault-circuit resistances in ohms for the 6 figures.
    Rf = Rfaults(id-55);                % Fault resistance for this figure.
    Lf = 5e-3;                          % Fault-loop inductance in henries (5 mH).
    t = linspace(0, 20e-3, 1800);       % 1800 time points from 0 to 20 ms (one 50 Hz cycle).
    i = (Vm/Rf)*(1-exp(-t*Rf/Lf));      % Fault current after a sudden DC step of Vm volts into R-L (not a sinusoidal fault).
    i2t = cumtrapz(t, i.^2);            % Running integral of current squared over time (I^2*t), in A^2*s.
    plot(ax, t*1e3, i2t, 'LineWidth', 2);  % Draw I^2*t against time in milliseconds.
    plotTitle = sprintf('Prospective fault I^2t | Rfault = %.1f ohm', Rf); % Title showing the fault resistance.
    notes = 'Simplified DC-step R-L fault, 325 V step, L=5 mH; not a sinusoidal AC fault simulation.'; % Note about the simplification.
else                                    % GROUP 9 (figures 62-65): extinction angle versus firing angle.
    Rvals = [10 15 20 30];              % Four load resistances in ohms, one per figure.
    Rtest = Rvals(id-61);               % Resistance for this figure.
    alphaDegrees = 5:5:165;             % Firing angles to test: 5 to 165 degrees in steps of 5.
    betaDegrees = zeros(size(alphaDegrees)); % Pre-allocate the array of extinction angles.
    for j = 1:numel(alphaDegrees)       % Loop over each firing angle.
        a = alphaDegrees(j)*pi/180;     % This firing angle in radians.
        theta = linspace(a, a+2*pi, 6000);  % 6000 angle points from firing to one full cycle later.
        current = ahdsRLCurrent(theta, a, Vm, Rtest, L, w); % SCR current for this firing angle and resistance.
        positive = find(current>1e-7);  % Indices where the current is clearly above zero.
        if isempty(positive)            % Check whether the SCR never conducts.
            betaDegrees(j) = NaN;       % Mark the extinction angle as undefined.
        else                            % Otherwise the SCR conducts.
            betaDegrees(j) = theta(positive(end))*180/pi; % Extinction angle in degrees: angle of the last conducting sample.
        end                             % End of the conduction check.
    end                                 % End of the loop over firing angles.
    plot(ax, alphaDegrees, betaDegrees, 'LineWidth', 2); % Draw extinction angle against firing angle.
    plot(ax, alphaDegrees, 180*ones(size(alphaDegrees)), '--'); % Draw a dashed line at 180 degrees (where the supply voltage crosses zero).
    legend(ax, 'Extinction angle beta', '180 deg voltage zero', 'Location', 'best'); % Add a legend naming the two lines.
    plotTitle = sprintf('R-L extinction sensitivity | R = %g ohm', Rtest); % Title showing the resistance.
    notes = 'Beta numerically detected from analytical R-L current. L=100 mH; isolated firing event.'; % Note about the method.
end                                     % End of the group selection.
hold(ax, 'off');                        % Turn hold off on this axes.
end                                     % End of function ahdsPlotSCR.


%% Analytical R-L load model for one isolated SCR firing event.
function current = ahdsRLCurrent(theta, alpha, Vm, R, L, w)
% INPUTS: theta and alpha in radians, Vm peak volts, R ohms, L henries, w rad/s.
% OUTPUT: current that is zero before firing, follows the R-L solution after firing,
%         and is forced to zero after the first natural current zero.
% Assumes i(alpha)=0 and no further gate firing during this interval.
Z = hypot(R, w*L);                      % Impedance magnitude of the R-L load in ohms.
phi = atan2(w*L, R);                    % Load phase angle in radians.
tau = L/R;                              % Electrical time constant L/R in seconds.
current = zeros(size(theta));           % Start with zero current at every angle.
active = theta >= alpha;                % True for angles at or after the firing angle.
th = theta(active);                     % The angles at or after firing.
raw = (Vm/Z)*(sin(th-phi)-sin(alpha-phi)*exp(-(th-alpha)/(w*tau))); % Analytical R-L current: steady-state sine minus the decaying transient.
firstPositive = find(raw>1e-8,1,'first');  % Index of the first sample where the current is clearly positive.
if isempty(firstPositive)               % Check whether the current is never positive.
    return;                             % Leave the current at zero and exit.
end                                     % End of the no-conduction check.
firstNonpositive = find(raw(firstPositive+1:end)<=0,1,'first'); % Position (counted after firstPositive) of the first sample with current at or below zero.
if ~isempty(firstNonpositive)           % Check whether the current falls to zero within the angle range.
    stopIndex = firstPositive+firstNonpositive-1; % Index of the last positive sample before extinction.
    raw(stopIndex+1:end) = 0;           % Force zero after extinction so the SCR cannot conduct again.
end                                     % End of the extinction check.
current(active) = max(raw,0);           % Store the result for angles after firing, with negative values set to zero.
end                                     % End of function ahdsRLCurrent.
```

```
%% Linear series R-L-C initial-energy discharge model.
function [vc, ic] = ahdsRLCTransient(t, V0, R, L, C)
% INPUTS: time vector in seconds, initial capacitor voltage V0, series R, L, C.
% OUTPUTS: capacitor voltage vc and series current ic at each time.
% Equations: dVc/dt = -i/C and di/dt = (Vc - R*i)/L.
% Method: eigen-decomposition of the 2x2 state matrix (exact solution of the linear equations).
A = [0 -1/C; 1/L -R/L];                 % State matrix A for the state vector [Vc; i].
x0 = [V0;0];                            % Initial state: capacitor at V0, zero current.
[V,D] = eig(A);                         % Eigenvectors V and eigenvalues D of A.
coeff = V\x0;                           % Coefficients of the initial state in the eigenvector basis.
modal = exp(diag(D)*t(:).') .* coeff;       % Each mode decays/oscillates as exp(eigenvalue*t), for all times at once.
states = real(V*modal);                 % Convert back to [Vc; i] for every time; keep the real part (imaginary part is numerical noise).
vc = reshape(states(1,:), size(t));         % First row is the capacitor voltage, reshaped like t.
ic = reshape(states(2,:), size(t));         % Second row is the current, reshaped like t.
end                                     % End of function ahdsRLCTransient.


%% Axis labels for the 65 distinct plot types.
function s = ahdsXLabel(id)
% Return the horizontal-axis label (with units) for figure number id.
if id <= 6                              % Figures 1-6: V-I curves.
   s = 'On-state voltage (V)';          % X axis is voltage.
elseif id <= 13                         % Figures 7-13: gate and turn-on.
   s = 'Time (microseconds)';           % X axis is time in microseconds.
elseif id <= 31                         % Figures 14-31: AC waveforms versus angle.
   s = 'Electrical angle (degrees)';    % X axis is electrical angle.
elseif id <= 41                         % Figures 32-41: R-L-C transients.
   s = 'Time (microseconds)';           % X axis is time in microseconds.
elseif id <= 48                         % Figures 42-48: turn-off margin.
   s = 'Available reverse-bias time (microseconds)'; % X axis is the available reverse-bias time.
elseif id <= 55                         % Figures 49-55: thermal.
   s = 'Time (seconds)';                % X axis is time in seconds.
elseif id <= 61                         % Figures 56-61: fault I^2*t.
   s = 'Time (milliseconds)';           % X axis is time in milliseconds.
else                                    % Figures 62-65: extinction sensitivity.
   s = 'Firing angle alpha (degrees)';  % X axis is the firing angle.
end                                     % End of the label selection.
end                                     % End of function ahdsXLabel.


function s = ahdsYLabel(id)
% Return the vertical-axis label (with units) for figure number id.
if id <= 6                              % Figures 1-6: V-I curves.
   s = 'Anode current (A)';             % Y axis is anode current.
elseif id <= 13                         % Figures 7-13: gate and turn-on.
   s = 'Normalized amplitude (unitless)';  % Y axis is a normalized value.
elseif id <= 21                         % Figures 14-21: resistive-load control.
   s = 'Voltage (V)';                   % Y axis is voltage.
elseif id <= 31                         % Figures 22-31: R-L load.
   s = 'Current (A)';                   % Y axis is current.
elseif id <= 41                         % Figures 32-41: R-L-C transients.
   s = 'Voltage (V)';                   % Y axis is voltage.
elseif id <= 48                         % Figures 42-48: turn-off margin.
   s = 'Turn-off margin (microseconds)';   % Y axis is the margin in microseconds.
elseif id <= 55                         % Figures 49-55: thermal.
   s = 'Junction temperature (deg C)';  % Y axis is temperature.
elseif id <= 61                         % Figures 56-61: fault I^2*t.
   s = 'Accumulated I^2t (A^2 s)';      % Y axis is accumulated I^2*t.
else                                    % Figures 62-65: extinction sensitivity.
   s = 'Extinction angle beta (degrees)';  % Y axis is the extinction angle.
end                                     % End of the label selection.
end                                     % End of function ahdsYLabel.


%% Folder helper: returns the first folder where a test file can really be written.
function folder = ahdsWritableFolder()
% OUTPUT: path of the first candidate folder in which a file can be created.
candidates = {pwd, '/MATLAB Drive', userpath, getenv('HOME'), tempdir}; % Candidate folders, in order of preference.
for k = 1:numel(candidates)             % Loop over the candidates.
   c = candidates{k};                   % Current candidate path.
   if isempty(c) || ~isfolder(c)        % Check whether it is empty or not an existing folder.
      continue;                         % Skip it and try the next one.
   end                                  % End of the existence check.
   testFile = fullfile(c, 'ahds_write_test.tmp'); % Path of a small test file in this folder.
   tfid = fopen(testFile, 'w');         % Try to create the test file (tfid < 0 means failure).
   if tfid >= 0                         % Check whether the file was created.
      fclose(tfid);                     % Close the test file.
      delete(testFile);                 % Delete the test file.
      folder = c;                       % Use this folder as the result.
      return;                           % Stop searching.
   end                                  % End of the write test.
end                                     % End of the loop over candidates.
error('AHDS:Output', 'No writable folder found. Move the script to MATLAB Drive and run it again.'); % No candidate worked: stop with a message.
end                                     % End of function ahdsWritableFolder.


%% Export helper: tries several PNG methods and checks the image really contains curves.
function method = ahdsExportFigure(fig, filepath, pauseSec)
```

```
% Tries each export method in turn. After each try, the PNG is read back and
% checked for coloured line pixels, so an image with only axes and text is
% rejected and the next method is used. Returns the name of the method that worked.
methods = {'print-painters', 'exportgraphics', 'print-opengl', 'getframe'}; % Export methods, tried in this order.
for m = 1:numel(methods)                % Loop over the methods.
   try                                  % Start a protected block: a failing method must not stop the others.
      if exist(filepath, 'file')        % Check whether a file from a previous attempt exists.
         delete(filepath);              % Delete it so it cannot pass the check by mistake.
      end                               % End of the old-file check.
      drawnow; pause(pauseSec);         % Finish drawing and wait a moment before exporting.
      switch methods{m}                 % Choose the code for the current method.
         case 'print-painters'          % Method 1: software renderer through print.
            set(fig, 'Renderer', 'painters'); % Select the painters renderer for this figure.
            print(fig, filepath, '-dpng', '-r300', '-painters'); % Save a 300 DPI PNG with painters.
         case 'exportgraphics'          % Method 2: exportgraphics.
            exportgraphics(fig, filepath, 'Resolution', 300); % Save a 300 DPI PNG.
         case 'print-opengl'            % Method 3: OpenGL renderer through print.
            set(fig, 'Renderer', 'opengl'); % Select the OpenGL renderer for this figure.
            print(fig, filepath, '-dpng', '-r300', '-opengl'); % Save a 300 DPI PNG with OpenGL.
         case 'getframe'                % Method 4: capture the figure as a screen image.
            set(fig, 'Visible', 'on'); drawnow; pause(pauseSec); % Show the figure (getframe needs it visible) and wait.
            fr = getframe(fig);         % Capture the figure as an image frame.
            imwrite(fr.cdata, filepath); % Write the captured pixels to the PNG file (screen resolution).
      end                               % End of the method selection.
      pause(pauseSec);                  % Wait so the file can finish being written.
      if ahdsHasCurve(filepath)         % Check that the PNG contains coloured line pixels.
         method = methods{m};           % Record which method worked.
         return;                        % Stop: this method produced a good image.
      end                               % End of the curve check.
   catch                                % If this method raised an error...
      % ...ignore it and try the next method.
   end                                  % End of the protected block.
end                                     % End of the loop over methods.
% No method passed the check: write one last image so the batch can still continue.
warning('AHDS:Unverified', 'Could not verify curves in %s; saved unverified.', filepath); % Warn the user about this image.
try                                     % Protected last attempt.
   exportgraphics(fig, filepath, 'Resolution', 300); % Try exportgraphics once more.
catch                                   % If that fails...
   print(fig, filepath, '-dpng', '-r300'); % ...fall back to print.
end                                     % End of the last attempt.
method = 'UNVERIFIED';                  % Report that the image could not be verified.
end                                     % End of function ahdsExportFigure.


function ok = ahdsHasCurve(filepath)
% True when the PNG has enough strongly coloured pixels to contain plot lines.
ok = false;                             % Assume there is no curve until proven otherwise.
info = dir(filepath);                   % Read the file information (size etc.).
if isempty(info) || info.bytes == 0     % Check whether the file is missing or empty.
   return;                              % Return false.
end                                     % End of the missing/empty check.
img = double(imread(filepath));         % Read the PNG pixels as numbers (may raise an error if the file is damaged).
if size(img, 3) < 3                     % Check whether the image has fewer than 3 colour channels.
   return;                              % Return false (cannot measure colour).
end                                     % End of the channel check.
sat = max(img, [], 3) - min(img, [], 3);    % Colourfulness of each pixel: grey, black and white give about 0, coloured lines give large values.
ok = nnz(sat > 60) > 3000;              % True if more than 3000 pixels are clearly coloured (so curves exist).
end                                     % End of function ahdsHasCurve.
```

Appendix B — Package map and references

01_MATLAB_Source/ — executable commented .m script

02_Reference_Figures_Illustrative/ — 65 pre-existing reference PNG images

03_Technical_Guide/ — this illustrated publication, shorter technical guide and figure index

04_Full_Reference/ — complete 250 Q&A FR/EN handbook with source code

05_Review_and_Provenance/ — change log and independent closure review

06_LinkedIn_Publication/ — English and French LinkedIn drafts

MathWorks: https://matlab.mathworks.com/ | https://www.mathworks.com/help/matlab/matlab_env/access-files-in-matlab-online.html | https://www.mathworks.com/help/matlab/ref/run.html